ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES



Plateforme agricole basée sur une architecture microservices

Matière :

Architecture Web Distribuée

Mini-projet – Validation 1

Réalisé par :

1. Nebras Soltani
2. Yosr Ben Ammar
3. Chayma Hammami
4. Meriam Bouzaffara
5. Maissa Nfissi
6. Mohamed Khaled Ben Kheder

Encadré par :

Oumeima IBN Elfekih

Classe :

DAS-I3 : grp4

Année universitaire 2026–2027

Table des matières

Chapitre 1: Présentation du projet **1**
1.1 Contexte et problématique . . . 1
1.1.1 Contexte général . . . 1
1.1.2 Problématique . . . 1
1.2 Solution proposée et objectifs . . . 2
1.2.1 Solution proposée . . . 2
1.2.2 Objectif général . . . 2
1.2.3 Objectifs spécifiques . . . 2
1.3 Acteurs et fonctionnalités principales . . . 2
1.3.1 Identification des acteurs . . . 3
1.3.2 Fonctionnalités principales . . . 3
1.4 Diagramme global de cas d'utilisation . . . 4
1.4.1 Présentation du diagramme . . . 4

Chapitre 2: Découpage en microservices **6**
2.1 Introduction . . . 6
2.2 Présentation générale des microservices . . . 6
2.2.1 Organisation des bases de données . . . 7
2.2.2 Technologies de communication . . . 8
2.3 MS1 – Gestion des utilisateurs . . . 8
2.3.1 Responsabilités . . . 8
2.3.2 Technologies utilisées . . . 9
2.3.3 Justification du choix de la base de données . . . 9
2.3.4 Fonctionnalités . . . 9
2.3.5 Entités principales . . . 10
2.3.5.1 Diagramme de classes . . . 11
2.3.6 Exemples d'API REST . . . 11
2.3.7 Interactions avec les autres microservices . . . 12
2.3.7.1 Services appelants et appelés . . . 12
2.3.7.2 Modes de communication et justification . . . 13
2.3.7.3 Comportement en cas d'indisponibilité . . . 13
2.4 MS2– Gestion les produits et ses offres et récoltes . . . 14

2.4.1 Responsabilité 14
2.4.2 Technologies utilisées 14
2.4.3 Choix de la base de données : H2 Database 14
2.4.4 Fonctionnalités 15
2.4.5 Description des principales entités et énumérations 15
2.4.6 Exemples d'API REST 16
2.4.7 Interactions avec les autres microservices 17
2.4.7.1 Services appelants et appelés 17
2.4.7.2 Mode de communication 18
2.4.7.3 Comportement prévu en cas d'indisponibilité 19
2.5 MS3– Gestion des commandes 20
2.5.1 Responsabilité 20
2.5.2 Technologies utilisées 20
2.5.3 Choix de la base de données : MySQL 20
2.5.4 Fonctionnalités 20
2.5.5 Description des principales entités et énumérations 21
2.5.6 Exemples d'API REST 22
2.5.7 Interactions avec les autres microservices 22
2.5.7.1 Services appelants et appelés 22
2.5.7.2 Mode de communication : REST synchrone 24
2.5.7.3 Comportement prévu en cas d'indisponibilité 24
2.6 MS4– Gestion des Livraisons 24
2.6.1 Responsabilité 24
2.6.2 Technologies utilisées 25
2.6.3 Choix de la base de données : MySQL 25
2.6.4 Fonctionnalités 25
2.6.5 Description des principales entités et énumérations 26
2.6.6 Cycle de vie d'une livraison 27
2.6.7 Exemples d'API REST 28
2.6.8 Interactions avec les autres microservices 29
2.6.8.1 Services appelants et appelés 29
2.6.8.2 Mode de communication 30
2.6.8.3 Règles métier et gestion des erreurs 30
2.7 MS5– Gestion des cooperatives 31
2.7.1 Responsabilité 31
2.7.2 Technologies utilisées 31
2.7.3 Choix de la base de données : MySQL 32
2.7.4 Fonctionnalités 32
2.7.5 Description des principales entités et énumérations 33

2.7.6 Exemples d'API REST 35
2.7.7 Interactions avec les autres microservices 35
2.7.7.1 Services appelants et appelés 35
2.7.7.2 Mode de communication 36
2.7.7.3 Comportement prévu en cas d'indisponibilité 37
2.8 MS6– Contrôle qualité et suivi des certifications agricoles 37
2.8.1 Responsabilité 37
2.8.2 Technologies utilisées 38
2.8.3 Choix de la base de données : H2 Database 38
2.8.4 Fonctionnalités 38
2.8.5 Description des principales entités et énumérations 39
2.8.6 Exemples d'API REST 40
2.8.7 Interactions avec les autres microservices 41
2.8.7.1 Services appelants et appelés 41
2.8.7.2 Mode de communication 42
2.8.7.3 Comportement prévu en cas d'indisponibilité 43
2.9 MS7– Gestion de la relation client 44
2.9.1 Responsabilité 44
2.9.2 Technologies utilisées 44
2.9.3 Choix de la base de données : MongoDB 44
2.9.4 Fonctionnalités 45
2.9.5 Description des principales entités 45
2.9.6 Exemples d'API REST 46
2.9.7 Interactions avec les autres microservices 47
2.9.7.1 Services appelants et appelés 47
2.9.7.2 Mode de communication 48
2.9.7.3 Comportement prévu en cas d'indisponibilité 48
2.10 Conclusion 49

Chapitre 3: Architecture technique et globale **50**
3.1 Introduction 50
3.2 Vue globale de l'architecture 50
3.2.1 Diagramme d'architecture globale 50
3.2.2 Principe général de fonctionnement 51
3.3 Description des composants techniques 52
3.3.1 Frontend Angular 52
3.3.2 API Gateway 52
3.3.3 Eureka Discovery Server 52
3.3.4 Spring Cloud Config Server 52

3.3.5 Keycloak : gestion de l'identité et de l'authentification 52
3.3.6 Microservices métier 53
3.4 Conteneurisation et déploiement 53
3.5 Gestion du code source et organisation du dépôt GitHub 54
3.5.1 Structure du dépôt 54
3.6 Conclusion 55

Table des figures

1.1 Diagramme de cas d'utilisation Globale 5

2.1 Diagramme de classes du microservice MS1 11
2.2 Diagramme de classes du microservice MS2 16
2.3 Diagramme d'interaction du microservice MS2 avec MS1, MS5 et MS6 18
2.4 Diagramme de classes du microservice MS3 21
2.5 Services qui échangent des informations avec MS3 23
2.6 Diagramme de classes du microservice MS4 27
2.7 Diagramme d'interaction du microservice MS4 avec MS1 et MS3 29
2.8 Diagramme de classes du microservice MS5 34
2.9 Diagramme d'interaction du microservice MS5 avec MS1 et MS2 36
2.10 Diagramme de classes du microservice MS6 40
2.11 Diagramme d'interaction du microservice MS6 avec MS1 et MS2 42
2.12 Diagramme de classes du microservice MS7 46
2.13 Diagramme d'interaction du microservice MS7 avec MS1 et MS3 47

3.1 Architecture technique et globale de la plateforme AgriLink 51

Liste des tableaux

1.1 Acteurs de la plateforme AgriLink 3

2.1 Présentation générale des microservices AgriLink 7
2.7 Données échangées entre MS2 et les autres microservices 18
2.10 Interactions entre MS3 et les autres microservices 23
2.11 Transitions autorisées des statuts d'une livraison 28
2.14 Données échangées entre MS4 et les autres microservices 29
2.17 Données échangées entre MS5 et les autres microservices 36
2.20 Données échangées entre MS6 et les autres microservices 41
2.23 Données échangées entre MS7 et les autres microservices 47

Chapitre 1

Présentation du projet

1.1 Contexte et problématique

1.1.1 Contexte général

Le secteur agricole joue un rôle important dans notre quotidien. Cependant, la vente et la distribution des produits agricoles présentent plusieurs difficultés.

Les producteurs ont parfois du mal à commercialiser leurs produits et à atteindre les clients. De leur côté, les clients manquent d'informations sur l'origine et la qualité des produits.

La gestion des commandes, des livraisons et des coopératives nécessite également une meilleure organisation.

1.1.2 Problématique

Les principales difficultés rencontrées sont :

— Le manque de visibilité des producteurs.
— La difficulté de gestion des produits et des stocks.
— Le manque d'informations sur l'origine des produits.
— Le manque de coordination entre les différents acteurs.
— La difficulté de suivi des commandes et des livraisons.
— Le manque de transparence sur la qualité des produits.

Face à ces difficultés, nous proposons de répondre à la question suivante :

Comment faciliter la commercialisation des produits agricoles, améliorer leur distribution et assurer leur traçabilité grâce à une plateforme numérique ?

1.2 Solution proposée et objectifs

1.2.1 Solution proposée

Pour répondre à cette problématique, nous proposons **AgriLink**, une plateforme web dédiée au secteur agricole.

Elle met en relation les producteurs et les clients. Les producteurs peuvent publier leurs produits et gérer leurs stocks. Les clients peuvent consulter les produits et passer des commandes.

La plateforme permet également de gérer les livraisons, les coopératives, les contrôles qualité et les certifications.

AgriLink est développée selon une architecture microservices. Chaque microservice prend en charge une partie précise des fonctionnalités de la plateforme.

1.2.2 Objectif général

L'objectif principal d'AgriLink est de faciliter la vente et la distribution des produits agricoles grâce à une plateforme numérique simple et centralisée.

1.2.3 Objectifs spécifiques

Les objectifs spécifiques du projet sont :

— Faciliter la commercialisation des produits agricoles.
— Permettre la gestion des produits, des récoltes et des stocks.
— Simplifier le passage et le suivi des commandes.
— Organiser les livraisons.
— Gérer les contrôles qualité et les certifications.
— Faciliter la gestion des coopératives.
— Permettre aux clients de publier des avis et des réclamations.
— Assurer la gestion des utilisateurs et de leurs accès.

1.3 Acteurs et fonctionnalités principales

La plateforme AgriLink comporte plusieurs acteurs. Chaque acteur dispose de fonctionnalités selon son rôle.

1.3.1 Identification des acteurs

Table 1.1 – Acteurs de la plateforme AgriLink

Acteur	**Description**
Visiteur	Consulte et recherche les offres publiées sur la marketplace.
Client	Commerçant ou consommateur : passe des commandes, suit ses livraisons, publie des avis et des réclamations. Le commerçant peut aussi exprimer des besoins.
Agriculteur	Gère ses produits, ses offres et ses récoltes. Il propose des quantités issues de ses récoltes et consulte les commandes concernant ses produits.
Transporteur	Consulte les livraisons qui lui sont attribuées et met à jour leur statut.
Administrateur	Gère les utilisateurs, supervise la plateforme et consulte les statistiques.
Responsable de coopérative	Gère sa coopérative, publie des annonces de collecte et traite les propositions de récolte des agriculteurs.
Responsable qualité et certifications	Gère les contrôles qualité, les certifications et les rapports associés.

1.3.2 Fonctionnalités principales

Les principales fonctionnalités sont réparties selon les acteurs :

— Visiteur :
  — Consulter les offres.
  — Rechercher et filtrer les offres.
— Client :
  — Gérer son profil.
  — Consulter et rechercher les offres.
  — Passer une commande et suivre son statut.
  — Consulter l'historique de ses commandes et annuler une commande.
  — Suivre les livraisons et confirmer la réception.
  — Créer, consulter et annuler un besoin (commerçant).
  — Publier des avis et des réclamations.

- Agriculteur :
  - Gérer ses produits.
  - Créer, publier et retirer ses offres.
  - Déclarer et gérer ses récoltes, et signaler une récolte endommagée.
  - Proposer une quantité issue d'une récolte.
  - Consulter les commandes.
- Transporteur :
  - Consulter les livraisons attribuées.
  - Mettre à jour le statut des livraisons.
  - Signaler un échec de livraison.
- Administrateur :
  - Gérer les utilisateurs.
  - Affecter un transporteur à une livraison.
  - Traiter les réclamations.
  - Consulter les statistiques.
- Responsable de coopérative :
  - Gérer sa coopérative.
  - Prendre en charge un besoin et publier une annonce de collecte.
  - Accepter, accepter partiellement ou refuser une proposition de récolte.
  - Enregistrer les quantités réellement reçues et clôturer ou annuler une annonce.
- Responsable qualité et certifications :
  - Effectuer les contrôles qualité.
  - Gérer les certifications.
  - Générer et exporter les rapports au format PDF.

1.4 Diagramme global de cas d'utilisation

Le diagramme global présente les acteurs et leurs interactions avec la plateforme AgriLink. Il regroupe les principales fonctionnalités du système.

1.4.1 Présentation du diagramme

La figure suivante représente le diagramme global de cas d'utilisation d'AgriLink.

Ce diagramme donne une vue générale des fonctionnalités d'AgriLink. Le chapitre suivant présente leur répartition entre les différents microservices.

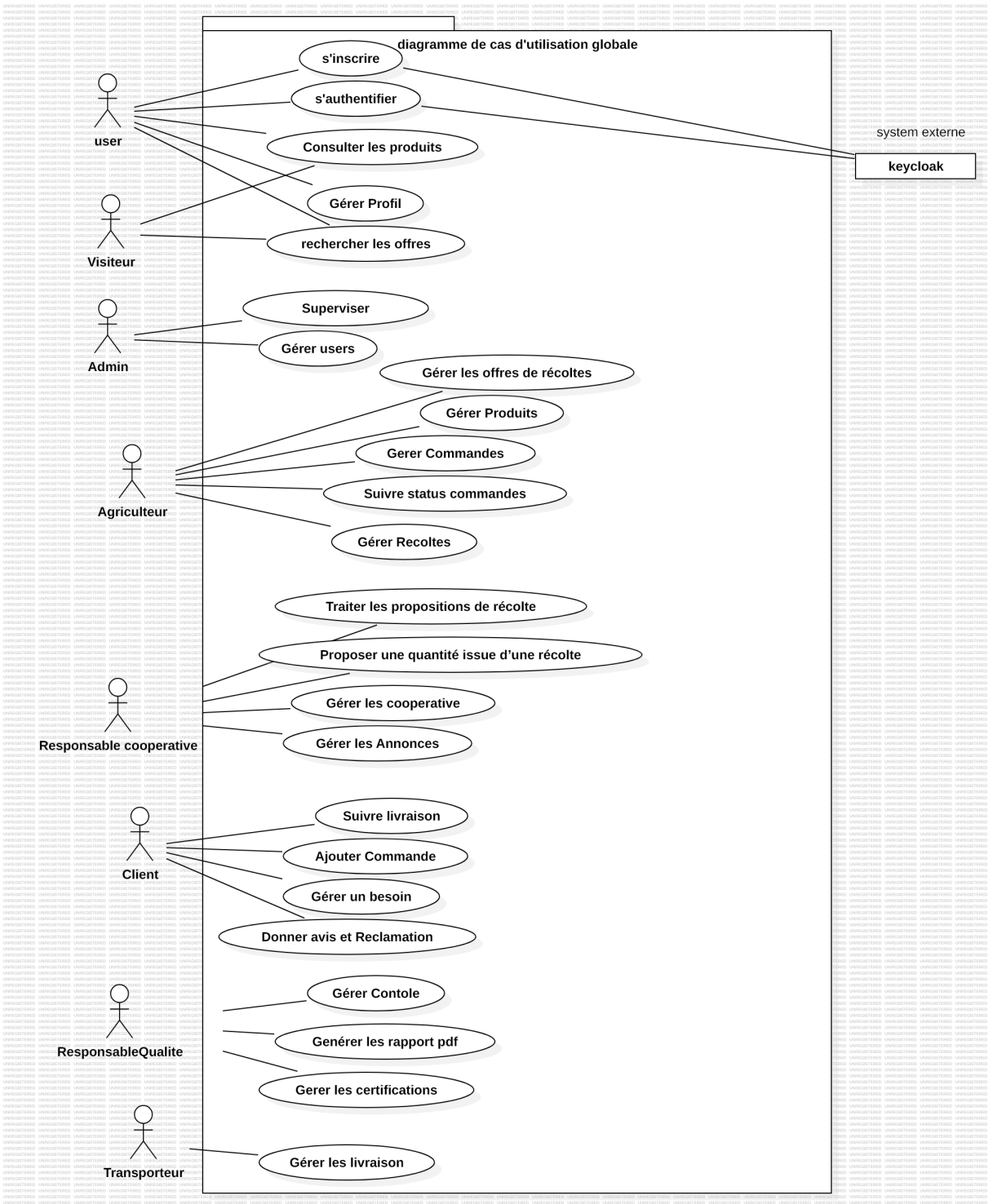


FIGURE 1.1 – Diagramme de cas d'utilisation Globale

CHAPITRE 2

DÉCOUPAGE EN MICROSERVICES

2.1 Introduction

Dans le cadre du projet AgriLink, l'architecture monolithique a été remplacée par une architecture basée sur les microservices. Cette approche consiste à diviser l'application en plusieurs services indépendants, chacun étant responsable d'un domaine fonctionnel précis.

AgriLink est composé de sept microservices. Chaque microservice possède ses propres responsabilités, ses propres entités et sa propre base de données. Les échanges entre les services sont assurés par des API REST pour les communications synchrones et par RabbitMQ pour les communications asynchrones.

2.2 Présentation générale des microservices

L'architecture de la plateforme AgriLink est composée de sept microservices indépendants. Chaque microservice prend en charge un domaine fonctionnel précis et possède sa propre logique métier, ses propres entités et sa propre base de données.

Les microservices communiquent entre eux à travers des API REST pour les échanges synchrones et RabbitMQ pour les échanges asynchrones basés sur des événements métier.

Le tableau suivant présente les responsabilités et les technologies utilisées par chaque microservice.

Table 2.1 – Présentation générale des microservices AgriLink

ID	Microservice	Technologie	Base de données	Responsabilité principale
MS1	User Management	Spring Boot	H2	Gestion des profils utilisateurs et des informations personnelles. L'authentification et les identités sont gérées par Keycloak.
MS2	Product & Harvest Management	Spring Boot	H2	Gestion du catalogue des produits agricoles, des récoltes, des offres individuelles et de la Marketplace.
MS3	Order Management	Spring Boot	MySQL	Gestion des commandes des clients, des lignes de commande et du suivi de leur état.
MS4	Delivery Management	Spring Boot	MySQL	Gestion des livraisons, affectation des transporteurs et suivi des opérations de livraison.
MS5	Cooperative Management	Spring Boot	MySQL	Gestion des besoins clients, des annonces de collecte et des propositions de récolte des agriculteurs.
MS6	Quality & Certification Management	Spring Boot	H2	Gestion des contrôles qualité, des certifications agricoles et de la génération des rapports.
MS7	Customer Relations Management	Node.js,NestJS	MongoDB	Gestion des avis clients, des notes attribuées aux agriculteurs, des réclamations et de leur traitement.

2.2.1 Organisation des bases de données

Chaque microservice dispose de sa propre base de données, conformément au principe *Database per Service*. Le choix du système de gestion de base de données dépend des besoins spécifiques de chaque domaine métier.

— **MS1 : H2** pour stocker les profils des utilisateurs. L'identité et l'authentification sont gérées séparément par Keycloak.
— **MS2 : H2** pour gérer les produits agricoles, les récoltes et les offres.

— **MS3 : MySQL** (choix proposé) pour gérer les commandes et garantir la cohérence des transactions.
— **MS4 : MySQL** (choix proposé) pour gérer les livraisons et les informations des transporteurs.
— **MS5 : MySQL** pour gérer les besoins clients, les responsables de coopératives, les annonces de collecte et les propositions de récolte.
— **MS6 : H2** pour stocker les contrôles qualité, les certifications et les rapports.
— **MS7 : MongoDB** pour stocker les avis clients, les réclamations et les historiques des échanges.

L'indépendance des bases de données garantit qu'un microservice ne peut pas accéder directement aux tables d'un autre service. Les échanges de données sont réalisés exclusivement à travers les API REST ou les événements métier.

2.2.2 Technologies de communication

Deux modes de communication sont utilisés dans l'architecture AgriLink :

— **Communication synchrone :** les API REST, avec OpenFeign pour les appels entre les microservices Spring Boot. Ce mode permet d'obtenir une réponse immédiate lorsqu'une vérification est nécessaire.
— **Communication asynchrone :** RabbitMQ, utilisé pour transmettre des événements métier sans attendre le traitement du service destinataire.

2.3 MS1 – Gestion des utilisateurs

Le microservice MS1 – User Management est responsable de la gestion des profils des utilisateurs de la plateforme AgriLink. Il concerne les agriculteurs, clients, transporteurs, responsables de coopérative, responsables qualité et certifications, administrateurs.

2.3.1 Responsabilités

MS1 est responsable de la gestion des profils et du stockage des informations personnelles dans sa base de données H2. Il permet aux utilisateurs de consulter et de modifier leurs profils. Il fournit également les informations nécessaires aux autres microservices. Keycloak assure la gestion des comptes, des mots de passe, de l'authentification et des rôles. Spring Security vérifie les jetons JWT et protège les API de MS1.

2.3.2 Technologies utilisées

Les technologies utilisées pour le développement de MS1 sont :

— **Java 17** : langage de programmation.
— **Spring Boot** : développement du microservice.
— **Spring Web (REST)** : création des API REST.
— **Spring Data JPA / Hibernate** : gestion de la persistance.
— **H2** : système de gestion de base de données.
— **Spring Security** : sécurisation des API.
— **Keycloak** : gestion des identités et de l'authentification.
— **Maven** : gestion des dépendances et compilation.
— **Postman** : test des API REST.
— **Docker** : conteneurisation du microservice.

2.3.3 Justification du choix de la base de données

La base de données H2 a été choisie pour MS1 pour les raisons suivantes :

— Base de données relationnelle légère et simple à configurer.
— Compatible avec Spring Data JPA et Hibernate.
— Permet de définir des contraintes d'unicité sur l'identifiant Keycloak (`keycloakId`).
— Adaptée au stockage des informations structurées des profils.
— Le mode fichier permet de conserver les données après le redémarrage de l'application.
— Chaque microservice possède sa propre base de données indépendante. Celle de MS1 est nommée `user_db`.

2.3.4 Fonctionnalités

Les principales fonctionnalités de MS1 sont :

— Inscription et création d'un profil utilisateur.
— Connexion et authentification via Keycloak.
— Consultation et modification du profil personnel.
— Consultation de la liste des utilisateurs par l'administrateur.
— Recherche d'un utilisateur par son nom ou son prénom.

— Ajout et suppression des utilisateurs par l'administrateur.
— Activation et désactivation d'un profil.
— Consultation des statistiques des utilisateurs.

Remarque : l'inscription implique la création d'une identité dans Keycloak, suivie de la création du profil correspondant dans MS1. L'authentification est entièrement assurée par Keycloak.

2.3.5 Entités principales

Le diagramme présente la classe principale `User`, qui représente le profil d'un utilisateur dans AgriLink. Elle contient les informations personnelles : identifiant, nom, prénom, adresse, identifiant Keycloak et dates de création et de modification.

Les principales classes représentées sont :

— **User :** représente le profil général de l'utilisateur. Elle contient les informations communes à tous les utilisateurs.
— **Admin :** permet de consulter, rechercher, ajouter et supprimer les utilisateurs.
— **Agriculteur :** contient les informations relatives à l'exploitation agricole et permet de consulter les avis.
— **Client :** représente un client qui peut consulter l'historique de ses commandes.
— **Transporteur :** contient les informations relatives à la disponibilité, à la zone de livraison et au permis.
— **ResponsableCooperative :** représente le responsable d'une coopérative et permet de gérer ses membres.
— **ResponsableQualite et certification :** intervient dans la gestion des certifications et la réalisation des contrôles.
— **KeycloakAccount :** représente le compte externe géré par Keycloak. Il est associé au profil `User` grâce à son identifiant.

Les classes spécialisées héritent de la classe `User`. Cette organisation permet de regrouper les informations communes et de représenter les fonctionnalités propres à chaque type d'utilisateur.

Remarque : `KeycloakAccount` est un élément externe et n'est pas une entité enregistrée dans la base H2. Keycloak assure l'authentification, la gestion des mots de passe et des rôles. MS1 conserve uniquement les informations personnelles des profils.

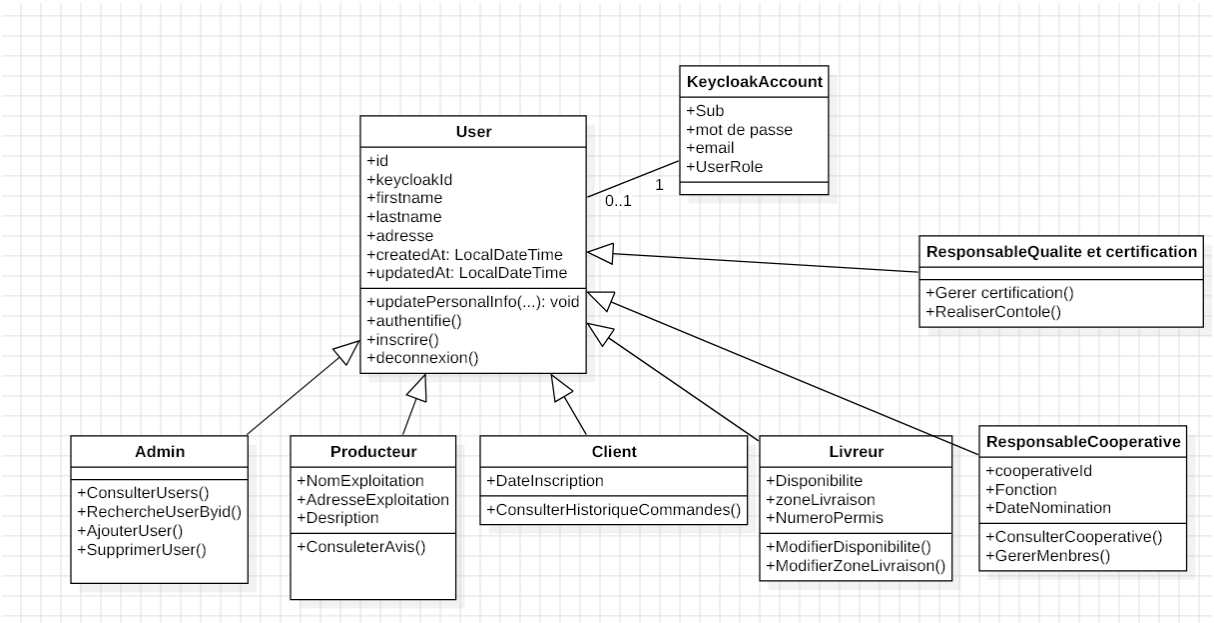


FIGURE 2.1 – Diagramme de classes du microservice MS1

2.3.5.1 Diagramme de classes

La figure suivante présente la structure des principales classes du microservice MS1 et leur relation avec le compte Keycloak.

2.3.6 Exemples d'API REST

MS1 expose plusieurs API REST pour permettre la gestion des profils utilisateurs.

HTTP	**Endpoint**	**Accès**	**Description**	**Codes**
POST	`/api/users`	Authentifié	Créer son profil après inscription (le `sub` vient du JWT).	201, 400, 401, 409
GET	`/api/users/{id}`	Authentifié	Consulter son profil.	200, 401, 404
PUT	`/api/users/{id}`	Authentifié	Modifier ses informations personnelles.	200, 400, 401, 404
GET	`/api/users`	ADMIN	Consulter la liste paginée des profils.	200, 401, 403
GET	`/api/users/{id}`	ADMIN	Consulter un profil par identifiant.	200, 401, 403, 404

GET	`/api/users/search?name=`	ADMIN	Rechercher un utilisateur par nom ou prénom.	200, 400, 401, 403
POST	`/api/admin/users`	ADMIN	Ajouter un utilisateur (compte Keycloak et profil).	201, 400, 403, 409, 503
DELETE	`/api/users/{id}`	ADMIN	Supprimer un utilisateur.	204, 403, 404, 503
GET	`/api/users/statistics`	ADMIN	Consulter les statistiques des utilisateurs.	200, 403

2.3.7 Interactions avec les autres microservices

MS1 échange des informations avec les six autres microservices. Il fournit les informations de profil nécessaires à leurs traitements métier.

2.3.7.1 Services appelants et appelés

Le tableau suivant présente les interactions proposées :

Microservice	**Sens**	**Données échangées**	**Mode**
MS2 – Agricultural Management	MS2 → MS1	Identifiant du producteur et informations de son profil.	REST / OpenFeign
MS3 – Order Management	MS3 → MS1	Identifiant du client, nom et adresse nécessaires à la commande.	REST / OpenFeign
MS4 – Delivery Management	MS4 → MS1	Identifiant du destinataire et informations nécessaires à la livraison.	REST / OpenFeign
MS5 – Cooperative Management	MS5 → MS1	Identifiants des membres et informations de leur profil.	REST / OpenFeign
MS6 – Quality Management	MS6 → MS1	Identifiant du producteur associé à un lot et informations nécessaires à la traçabilité.	REST / OpenFeign

MS7 – Customer Relations	MS7 → MS1	Identifiant de l'utilisateur, nom et prénom de l'auteur d'un avis.	REST / OpenFeign

2.3.7.2 Modes de communication et justification

Communication synchrone :

MS1 utilise principalement une communication synchrone avec les autres microservices à travers les API REST et OpenFeign. Elle permet à un service de demander immédiatement les informations d'un utilisateur et de recevoir une réponse.

Par exemple, MS3 peut demander à MS1 les informations du client associé à une commande.

Communication asynchrone :

MS1 peut également utiliser RabbitMQ pour publier des événements métier, tels que :

— `UserRegistered` : création d'un nouveau profil.
— `UserProfileUpdated` : modification d'un profil.
— `UserDeleted` : suppression d'un profil.

Cette communication permet aux autres microservices de recevoir les événements sans attendre une réponse immédiate.

Justification du choix :

La communication synchrone est adaptée lorsque le service appelant a besoin d'une réponse immédiate. La communication asynchrone est adaptée pour informer les autres services d'un événement sans les bloquer.

2.3.7.3 Comportement en cas d'indisponibilité

Les comportements prévus en cas de panne sont les suivants :

— **MS1 indisponible :** le service appelant reçoit une erreur contrôlée. Des mécanismes de timeout et de circuit breaker peuvent être utilisés.
— **Keycloak indisponible :** les nouvelles authentifications et les opérations nécessitant la création ou la suppression d'un compte sont temporairement impossibles. Les JWT déjà émis peuvent rester valides jusqu'à leur expiration, selon la disponibilité des clés publiques.

— **RabbitMQ indisponible :** les événements peuvent être conservés temporairement puis publiés ultérieurement grâce à un mécanisme Outbox.
— **Profil inexistant :** MS1 retourne une réponse HTTP 404 (Not Found).

Conclusion : MS1 constitue le point de référence pour les profils utilisateurs. Il permet aux autres microservices d'accéder aux informations nécessaires tout en respectant la séparation des responsabilités.

2.4 MS2– Gestion les produits et ses offres et récoltes

2.4.1 Responsabilité

MS2 assure la gestion du catalogue des produits agricoles, des récoltes et des offres individuelles. Il permet aux agriculteurs de créer des offres pour des produits existants ou d'ajouter de nouveaux produits, puis de publier leurs offres sur la Marketplace. Il permet également aux clients et aux visiteurs de consulter et de rechercher les offres disponibles.

2.4.2 Technologies utilisées

Les technologies utilisées pour le développement de MS1 sont :

— **Java 17** : langage de programmation.
— **Spring Boot** : développement du microservice.
— **Spring Web (REST)** : création des API REST.
— **Spring Data JPA / Hibernate** : gestion de la persistance.
— **H2** : système de gestion de base de données.
— **Spring Security** : sécurisation des API.
— **Maven** : gestion des dépendances et compilation.
— **Postman** : test des API REST.
— **Docker** : conteneurisation du microservice.

2.4.3 Choix de la base de données : H2 Database

MS2 utilise H2, une base de données relationnelle légère et compatible avec SQL. Ce choix est justifié par les raisons suivantes :

— Facile à configurer et à intégrer avec Spring Boot.
— Permet de gérer les relations entre les entités `Product`, `Recolte` et `Offer`.

— Supporte les transactions SQL afin de garantir la cohérence des données.
— Adaptée aux phases de développement et de test grâce à sa légèreté et sa simplicité d'utilisation.

2.4.4 Fonctionnalités

1. **Gestion des produits – Agriculteur**
   — Consulter les produits existants.
   — Ajouter et gérer les produits.
2. **Gestion des récoltes – Agriculteur**
   — Déclarer et gérer ses récoltes.
   — Suivre les quantités et la disponibilité.
   — Signaler une récolte endommagée ou indisponible.
3. **Gestion des offres – Agriculteur**
   — Créer une offre pour un produit existant ou nouvellement créé.
   — Gérer ses offres.
   — Publier ou retirer une offre de la Marketplace.
4. **Gestion de la Marketplace – Client / Visiteur**
   — Consulter les offres publiées.
   — Rechercher, filtrer et trier les offres.
   — Consulter les détails des offres et les informations des vendeurs.

2.4.5 Description des principales entités et énumérations

Le microservice MS2 – Product & Harvest Management repose sur les entités et les énumérations suivantes :

— **Product :** Représente un produit agricole enregistré dans le catalogue par un agriculteur.
— **Recolte :** Représente la production récoltée par un agriculteur, avec ses quantités, ses dates et sa disponibilité.
— **Offer :** Représente une offre commerciale publiée sur la Marketplace pour vendre un produit agricole.
— **RecolteStatus :** Définit les états possibles d'une récolte : disponible, partiellement disponible, endommagée ou indisponible.
— **OfferStatus :** Définit les états possibles d'une offre : brouillon, publiée, retirée ou archivée.

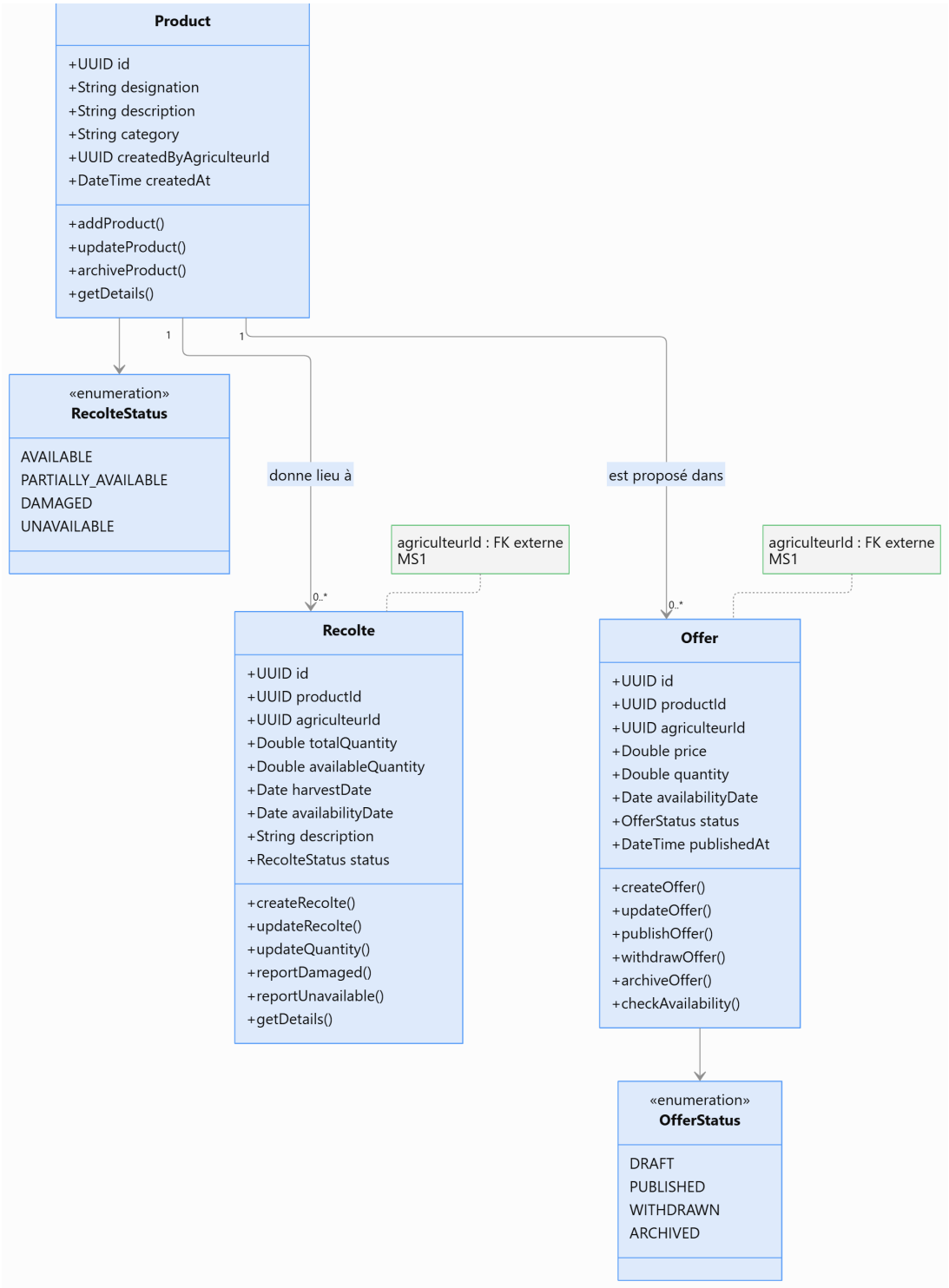


Figure 2.2 – Diagramme de classes du microservice MS2

2.4.6 Exemples d'API REST

Le système expose plusieurs API REST permettant la gestion des produits, la déclaration et le suivi des récoltes, ainsi que la création et la consultation des offres publiées sur la plateforme.

Méthode	Endpoint	Rôle / usage	Code HTTP
GET	`/api/products`	Consulter les produits existants.	200 OK
GET	`/api/products/{id}`	Consulter les détails d'un produit.	200 OK, 404 Not Found
POST	`/api/products/{productId}/recoltes`	Déclarer une récolte associée à un produit.	201 Created, 404 Not Found
PATCH	`/api/recoltes/{id}/status`	Modifier le statut d'une récolte.	200 OK, 400 Bad Request
POST	`/api/offers`	Créer une offre pour un produit existant.	201 Created, 400 Bad Request
PUT	`/api/offers/{id}`	Modifier les informations d'une offre.	200 OK, 404 Not Found
GET	`/api/marketplace/offers`	Consulter et rechercher les offres publiées.	200 OK

2.4.7 Interactions avec les autres microservices

MS2 – Agricultural Management interagit avec MS1, MS5 et MS6 pour vérifier les profils des agriculteurs, gérer les réservations des récoltes et assurer le suivi des certifications.

2.4.7.1 Services appelants et appelés

Le tableau suivant présente les données échangées entre MS2 et les autres microservices :

Microservice	Sens	Données échangées	Mode
MS1 – User Management	MS2 → MS1	Identifiant `agriculteurId`. MS1 retourne l'existence du profil, son statut et son rôle.	REST / OpenFeign
MS5 – Cooperative Management	MS5 → MS2	`productId`, `recolteId`, `agriculteurId` et quantité proposée. MS2 retourne les informations de la récolte et confirme ou refuse la réservation.	REST / OpenFeign
MS5 – Cooperative Management	MS5 → MS2	Événement `QuantiteRecolteRecue` contenant `recolteId` et `quantiteRecue`.	RabbitMQ
MS6 – Quality Management	MS6 → MS2	Identifiant `recolteId`. MS2 retourne les informations nécessaires concernant la récolte.	REST / OpenFeign
MS6 – Quality Management	MS6 → MS2	Événement `CertificationIssued` contenant `certificationId`, `recolteId`, `statut`, `dateDelivrance` et `dateExpiration`.	RabbitMQ

TABLE 2.7 – Données échangées entre MS2 et les autres microservices

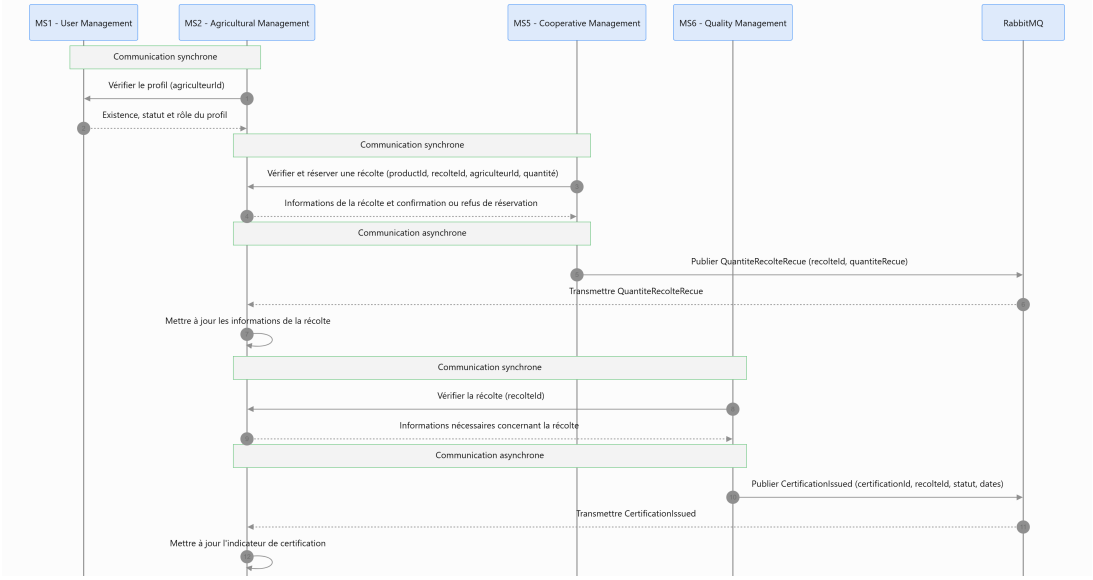


FIGURE 2.3 – Diagramme d'interaction du microservice MS2 avec MS1, MS5 et MS6

2.4.7.2 Mode de communication

Communication synchrone : REST avec OpenFeign

MS2 utilise une communication REST synchrone avec MS1 pour vérifier l'existence et le statut du profil de l'agriculteur avant certaines opérations métier.

MS5 et MS6 utilisent également REST avec OpenFeign pour interroger MS2 :

— MS5 vérifie les produits et les récoltes, leur propriétaire et leur disponibilité, puis demande la réservation des quantités.

— MS6 vérifie l'existence d'une récolte avant d'effectuer un contrôle qualité ou de lui associer une certification.

Cette communication permet d'obtenir une réponse immédiate avant de réaliser les opérations concernées.

Communication asynchrone : RabbitMQ

MS2 consomme les événements métier publiés par MS5 et MS6 :

— `QuantiteRecolteRecue` : permet de mettre à jour les informations de la récolte après la réception effective d'une quantité.
— `CertificationIssued` : permet de mettre à jour l'indicateur de certification associé à la récolte.

Cette communication permet aux microservices d'échanger des informations sans attendre la fin du traitement de l'autre service.

2.4.7.3 Comportement prévu en cas d'indisponibilité

Les comportements prévus en cas de panne sont les suivants :

— **MS1 indisponible :** MS2 interrompt l'opération nécessitant la vérification du profil et retourne une erreur `503 - Service Unavailable` ou `504 - Gateway Timeout`. Des mécanismes de Retry et de Circuit Breaker peuvent être utilisés.
— **MS2 indisponible :** MS5 et MS6 interrompent les opérations nécessitant une réponse de MS2. Ils retournent une erreur `503 - Service Unavailable` ou `504 - Gateway Timeout`. Aucune réservation ou certification nécessitant une vérification ne doit être validée sans confirmation de MS2.
— **Profil inexistant :** MS1 retourne `404 - Not Found`. Si le rôle de l'utilisateur est insuffisant, l'opération est refusée avec `403 - Forbidden`.
— **RabbitMQ indisponible :** les événements sont conservés temporairement dans une table Outbox du microservice émetteur (MS5 ou MS6), puis republiés après le rétablissement du broker grâce aux mécanismes de Retry.
— **Échec du traitement d'un événement :** MS2 utilise des mécanismes de Retry, d'idempotence et une Dead Letter Queue (DLQ) afin de garantir un traitement fiable et d'éviter les doublons.

2.5 MS3– Gestion des commandes

2.5.1 Responsabilité

Le microservice MS3 gère les commandes passées sur AgriLink. Il prend en charge leur création, le calcul des montants, la confirmation, l'annulation et le suivi de leur état.

2.5.2 Technologies utilisées

Les technologies utilisées pour le développement de MS1 sont :

— **Java 17** : langage de programmation.
— **Spring Boot** : développement du microservice.
— **Spring Web (REST)** : création des API REST.
— **Spring Data JPA / Hibernate** : gestion de la persistance.
— **MySQL** : système de gestion de base de données.
— **Spring Security** : sécurisation des API.
— **Maven** : gestion des dépendances et compilation.
— **Postman** : test des API REST.
— **Docker** : conteneurisation du microservice.

2.5.3 Choix de la base de données : MySQL

Pour MS3, MySQL est proposé comme base de données relationnelle. Ce choix doit être confirmé dans le dépôt du microservice.

La justification de ce choix repose sur les raisons suivantes :

— Stocke les informations structurées des commandes et des lignes de commande.
— Permet de gérer la relation entre `Commande` et `LigneCommande` grâce à une clé étrangère.
— Facilite le calcul et la conservation des montants et des statuts des commandes.
— Prend en charge les transactions SQL afin de garantir la cohérence des données.

2.5.4 Fonctionnalités

1. **Création des commandes – Client**
   — Créer une commande contenant un ou plusieurs produits.

— Associer les produits, les quantités et les prix unitaires.

— Calculer le montant total à partir des sous-totaux des lignes de commande.

2. **Confirmation et annulation – Client**
   — Vérifier la disponibilité et demander la réservation du stock auprès de MS2.
   — Confirmer la commande si la réservation du stock réussit.
   — Annuler une commande lorsque son état le permet.
3. **Consultation et suivi – Client**
   — Consulter les détails d'une commande et son montant total.
   — Afficher l'historique des commandes du client.
   — Suivre le statut de la commande : `PENDING`, `CONFIRMED`, `IN_PREPARATION`, `SHIPPED`, `DELIVERED` ou `CANCELLED`.

2.5.5 Description des principales entités et énumérations

Le microservice MS3 – Order Management repose sur les entités et l'énumération suivantes :

— **Commande :** Représente une commande passée par un client. Elle contient son identifiant, celui du client, la date, le montant total et le statut.

— **LigneCommande :** Représente un produit appartenant à une commande, avec son identifiant, la quantité, le prix unitaire et le sous-total. Une commande contient une ou plusieurs lignes.

— **StatutCommande :** Définit les états possibles d'une commande : `PENDING`, `CONFIRMED`, `IN_PREPARATION`, `SHIPPED`, `DELIVERED` ou `CANCELLED`.

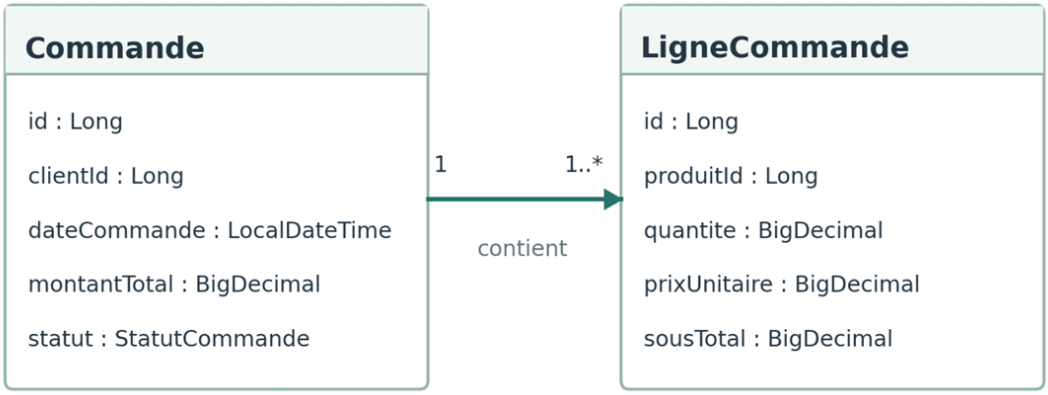


Figure 2.4 – Diagramme de classes du microservice MS3

2.5.6 Exemples d'API REST

Le microservice MS3 expose plusieurs API REST permettant la création, la confirmation, l'annulation, la consultation et le suivi des commandes.

Méthode	**Endpoint**	**Rôle / usage**	**Code HTTP**
POST	`/api/orders`	Créer une commande.	201 Created, 400 Bad Request
PATCH	`/api/orders/{id}/confirm`	Confirmer la commande après vérification du stock.	200 OK, 409 Conflict
PATCH	`/api/orders/{id}/cancel`	Annuler une commande selon son état.	200 OK, 409 Conflict
GET	`/api/orders/{id}`	Consulter les détails et le statut d'une commande.	200 OK, 404 Not Found
GET	`/api/orders/customer/{id}`	Consulter l'historique des commandes d'un client.	200 OK

2.5.7 Interactions avec les autres microservices

MS3 – Order Management échange des informations avec MS1, MS2, MS4 et MS7 afin d'assurer la gestion des commandes, la vérification du stock et la coordination avec les autres services.

2.5.7.1 Services appelants et appelés

Le tableau suivant présente les interactions entre MS3 et les autres microservices :

Microservice	Interaction	Mode
MS1 – User Management	Identifier le client, notamment à travers `clientId` présent dans le jeton d'authentification.	JWT
MS2 – Agricultural Management	Vérifier la disponibilité des produits et demander la réservation des quantités commandées.	REST synchrone
MS4 – Delivery Management	Être informé après la confirmation d'une commande, par exemple via l'événement `OrderConfirmed`.	RabbitMQ (envisagé)
MS7 – Customer Relations Management	Consulter une commande afin de vérifier son propriétaire et son état avant certaines opérations.	REST / OpenFeign (à confirmer)

Table 2.10 – Interactions entre MS3 et les autres microservices

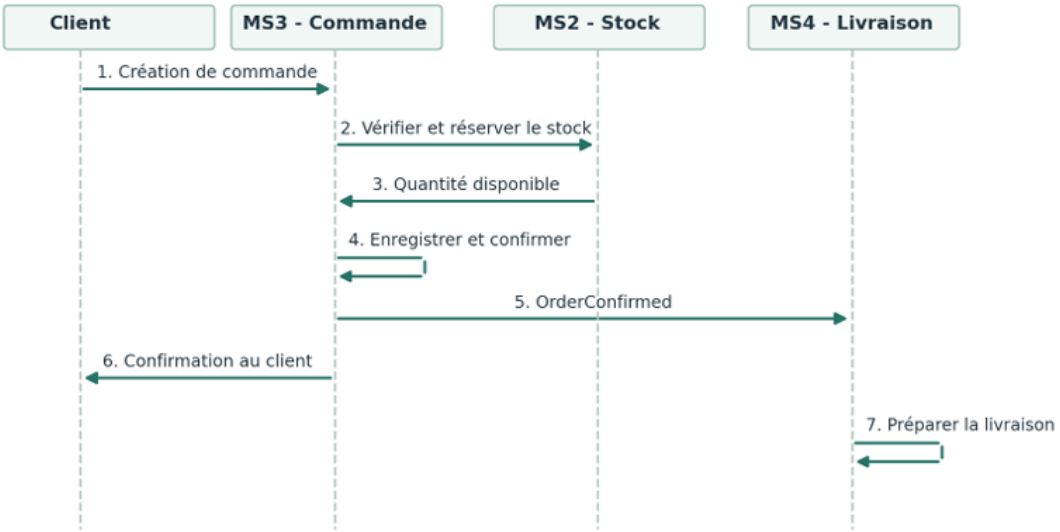


Figure 2.5 – Services qui échangent des informations avec MS3

Interaction avec MS2 – Vérification et réservation du stock

MS3 sollicite MS2 – Product & Harvest Management pour vérifier la disponibilité des produits et demander la réservation des quantités commandées.

Données échangées :

— MS3 transmet les références des produits (`produitId`) et les quantités demandées.

— MS2 retourne les informations nécessaires sur la disponibilité et le résultat de la réservation. Le prix unitaire utilisé pour le calcul provient des données de MS2.

2.5.7.2 Mode de communication : REST synchrone

La communication est synchrone, car MS3 doit obtenir la réponse de MS2 concernant la disponibilité et la réservation du stock avant de confirmer la commande.

L'utilisation d'OpenFeign est possible, mais reste à confirmer dans le code du microservice.

Communication avec MS4 – Livraison

Après la confirmation d'une commande, MS3 peut informer MS4 – Delivery Management afin de permettre la préparation de la livraison.

Un événement métier nommé `OrderConfirmed` peut être utilisé pour transmettre les informations nécessaires.

Cette communication asynchrone reste envisagée et dépend de l'architecture qui sera mise en place dans le dépôt du projet.

2.5.7.3 Comportement prévu en cas d'indisponibilité

Si MS2 est indisponible ou ne répond pas, MS3 peut effectuer un nombre limité de tentatives grâce au mécanisme de Retry.

En cas d'échecs répétés, un Circuit Breaker peut interrompre temporairement les appels afin de protéger les deux microservices.

Ces mécanismes sont proposés et restent à vérifier dans l'implémentation.

Dans ce cas :

— MS3 ne confirme pas une commande dont le stock n'a pas pu être vérifié ou réservé.
— L'API peut retourner une erreur `503 - Service Unavailable`, indiquant que la validation du stock est momentanément impossible.
— Le client est invité à réessayer ultérieurement.

2.6 MS4– Gestion des Livraisons

2.6.1 Responsabilité

MS4 assure l'organisation, l'affectation et le suivi des livraisons au sein de la plateforme AgriLink. Il permet de gérer les livraisons depuis la confirmation d'une commande jusqu'à sa réception par le client, tout en assurant le suivi de leur évolution.

2.6.2 Technologies utilisées

Les technologies utilisées pour le développement de MS4 sont :

— **Angular** : développement de l'interface utilisateur.
— **Spring Boot** : développement du microservice.
— **MySQL** : système de gestion de base de données.
— **API REST / OpenFeign** : communication synchrone avec les autres microservices.
— **RabbitMQ** : communication asynchrone par événements.
— **JWT** : authentification et gestion des autorisations.

Spring Boot assure la logique métier et la gestion des API REST. Angular permet aux utilisateurs d'interagir avec les fonctionnalités du microservice. MySQL stocke les informations relatives aux livraisons et à leur suivi.

RabbitMQ permet l'échange d'événements avec MS3, tandis qu'OpenFeign permet à MS4 d'interroger MS1 pour vérifier les informations des transporteurs avant leur affectation.

2.6.3 Choix de la base de données : MySQL

MS4 utilise MySQL comme système de gestion de base de données relationnelle. Ce choix est justifié par les raisons suivantes :

— Permet de stocker les informations des livraisons et des événements de suivi.
— Facilite la gestion des relations entre les entités `Livraison` et `TrackingEvent`.
— Assure la cohérence des données relatives aux livraisons et à leurs changements de statut.
— Permet de conserver les identifiants externes (`orderId`, `transporteurId` et `clientId`) sans dupliquer les données des autres microservices.

2.6.4 Fonctionnalités

1. **Gestion des livraisons – Administrateur**
   — Créer automatiquement une livraison pour une commande confirmée.
   — Affecter un transporteur à une livraison.
   — Enregistrer l'adresse et la date prévue de livraison.
   — Modifier ou annuler une livraison selon son statut.
2. **Suivi des livraisons – Transporteur**

- Modifier le statut d'une livraison.
- Enregistrer les différentes étapes du transport.
- Consulter les détails et l'historique du suivi.
- Signaler un échec de livraison.

3. **Réception des livraisons – Client**
   - Consulter les informations de livraison.
   - Suivre l'évolution de la livraison.
   - Confirmer la réception de la commande.

2.6.5 Description des principales entités et énumérations

Le microservice MS4 – Delivery Management repose sur les entités et l'énumération suivantes :

- **Livraison :** représente une livraison associée à une commande. Elle contient les identifiants du client, du transporteur, l'adresse, la date prévue et le statut.
- **TrackingEvent :** représente un événement permettant de suivre l'évolution du statut d'une livraison. Il contient la date, la description et l'identifiant de l'utilisateur ayant effectué le changement.
- **DeliveryStatus :** définit les différents statuts possibles d'une livraison : `PENDING`, `ASSIGNED`, `PICKED_UP`, `IN_TRANSIT`, `DELIVERED`, `FAILED` et `CANCELLED`.

Chaque livraison peut générer plusieurs événements de suivi. Chaque changement de statut crée automatiquement un nouvel objet `TrackingEvent`.

Les identifiants `orderId`, `transporteurId` et `clientId` sont des références externes vers les autres microservices et ne constituent pas des clés étrangères dans la base de données de MS4.

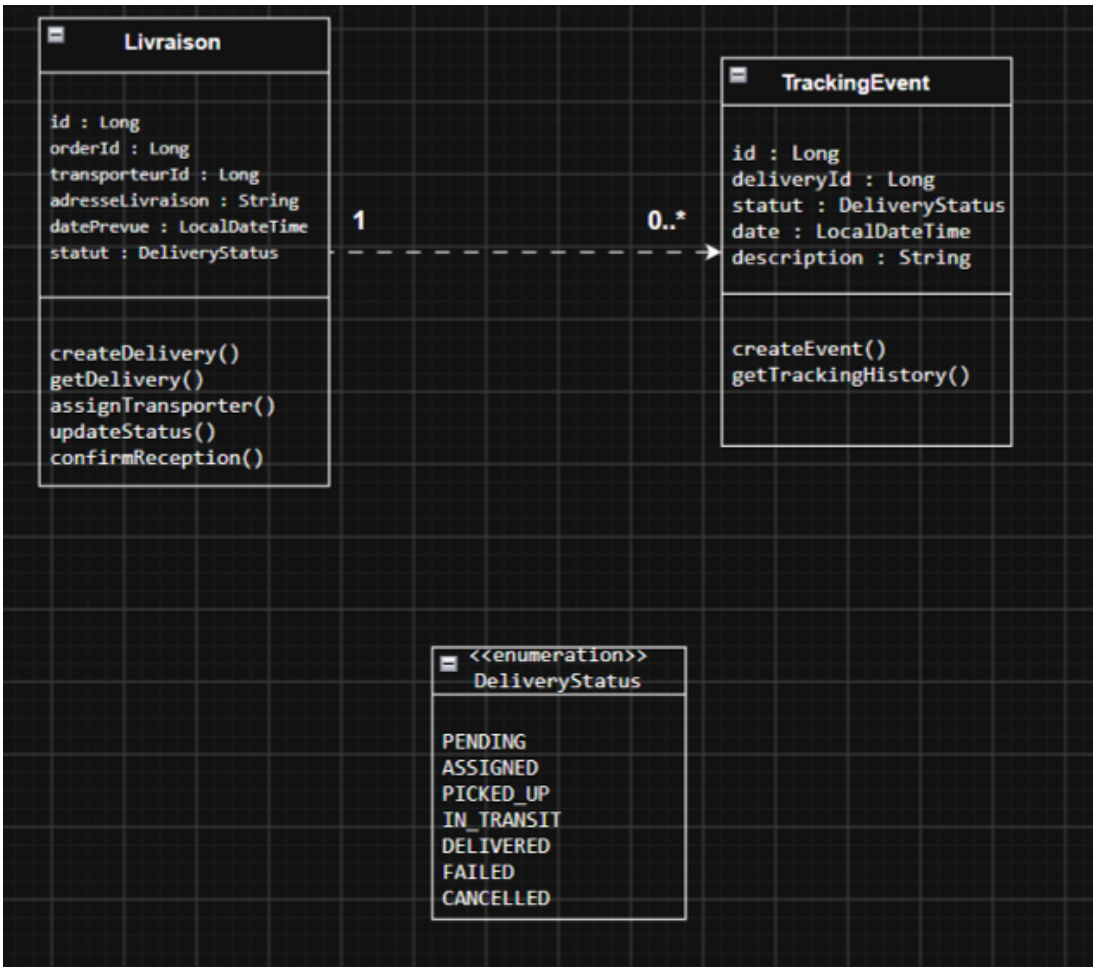


Figure 2.6 – Diagramme de classes du microservice MS4

2.6.6 Cycle de vie d'une livraison

Une livraison peut passer par les différents statuts suivants :

- **PENDING :** livraison créée et en attente d'affectation d'un transporteur.
- **ASSIGNED :** un transporteur a été affecté.
- **PICKED_UP :** la commande a été récupérée par le transporteur.
- **IN_TRANSIT :** la commande est en cours d'acheminement vers le client.
- **DELIVERED :** la commande a été reçue par le client.
- **FAILED :** la livraison a échoué.
- **CANCELLED :** la livraison a été annulée.

Les transitions autorisées sont présentées dans le tableau suivant :

Statut actuel	Statuts suivants autorisés
PENDING	ASSIGNED, CANCELLED
ASSIGNED	PICKED_UP, CANCELLED
PICKED_UP	IN_TRANSIT, FAILED
IN_TRANSIT	DELIVERED, FAILED
DELIVERED	Aucun (statut final)
FAILED	Aucun (statut final)
CANCELLED	Aucun (statut final)

Table 2.11 – Transitions autorisées des statuts d'une livraison

2.6.7 Exemples d'API REST

MS4 expose plusieurs API REST permettant de créer les livraisons, d'affecter les transporteurs, de modifier les statuts et de consulter l'historique du suivi.

Méthode	Endpoint	Rôle / usage
POST	`/api/deliveries`	Créer une livraison à partir d'une commande confirmée.
GET	`/api/deliveries/{id}`	Consulter les informations d'une livraison.
GET	`/api/deliveries/order/{orderId}`	Consulter la livraison associée à une commande.
PATCH	`/api/deliveries/{id}/assign`	Affecter un transporteur à une livraison.
PATCH	`/api/deliveries/{id}/status`	Modifier le statut d'une livraison.
GET	`/api/deliveries/{id}/tracking`	Consulter l'historique du suivi d'une livraison.

Remarque : la création d'une livraison est normalement déclenchée automatiquement par la réception de l'événement `OrderConfirmed` publié par MS3. L'API POST peut également être utilisée pour une création interne ou administrative contrôlée.

2.6.8 Interactions avec les autres microservices

MS4 – Delivery Management interagit principalement avec MS1 et MS3 pour vérifier les informations des transporteurs, recevoir les commandes confirmées et notifier l'évolution des livraisons.

2.6.8.1 Services appelants et appelés

Le tableau suivant présente les données échangées entre MS4 et les autres microservices :

Microservice	Sens	Données échangées	Mode
MS3 – Order Management	MS3 → MS4	Événement `OrderConfirmed` contenant `orderId`, `clientId`, l'adresse de livraison et la date souhaitée.	RabbitMQ
MS3 – Order Management	MS4 → MS3	Événements `DeliveryStatusChanged` et `DeliveryDelivered` contenant `deliveryId`, `orderId`, le statut et la date.	RabbitMQ
MS1 – User Management	MS4 → MS1	Identifiant `transporteurId`. MS1 vérifie l'existence du profil et retourne son statut (actif ou non) et son rôle.	REST / OpenFeign

Table 2.14 – Données échangées entre MS4 et les autres microservices

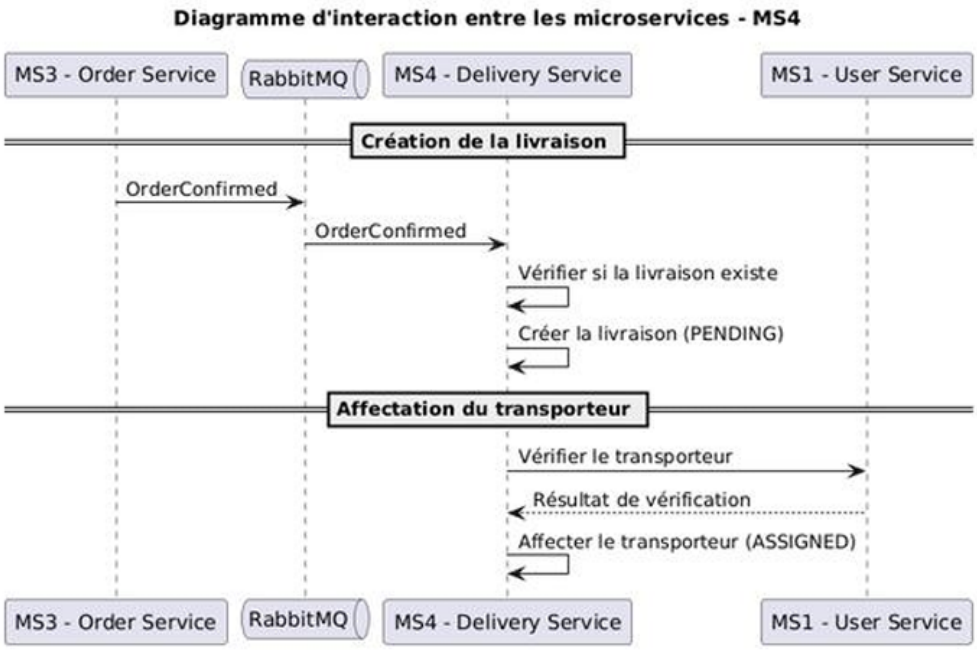


Figure 2.7 – Diagramme d'interaction du microservice MS4 avec MS1 et MS3

2.6.8.2 Mode de communication

Communication synchrone : REST avec OpenFeign

MS4 utilise une communication REST synchrone avec MS1 pour vérifier les informations du transporteur avant son affectation à une livraison.

Cette vérification permet de s'assurer que le transporteur :

— Existe dans la plateforme.
— Possède un profil actif.
— Dispose du rôle `TRANSPORTEUR`.

MS4 n'effectue l'affectation qu'après confirmation de ces informations par MS1.

Communication asynchrone : RabbitMQ

MS4 utilise RabbitMQ pour échanger des événements métier avec MS3 :

— `OrderConfirmed` : MS4 consomme cet événement publié par MS3 afin de créer automatiquement la livraison correspondante.
— `DeliveryStatusChanged` : MS4 publie cet événement lors des changements de statut afin de notifier MS3 de l'évolution de la livraison.
— `DeliveryDelivered` : lorsque la livraison atteint le statut `DELIVERED`, MS4 publie cet événement pour notifier la réception effective de la commande.

MS3 utilise ces événements pour mettre à jour le statut de la commande. Ainsi, lorsque la commande est récupérée par le transporteur, MS3 peut passer son statut à `SHIPPED`. Lorsqu'elle est livrée, MS3 la passe à `DELIVERED`.

Cette mise à jour permet également à MS7 de vérifier qu'une commande est livrée avant d'autoriser la publication d'un avis.

La communication asynchrone permet de limiter le couplage entre MS3 et MS4, car MS3 peut confirmer une commande sans dépendre de la disponibilité immédiate de MS4.

2.6.8.3 Règles métier et gestion des erreurs

Les principales règles métier appliquées par MS4 sont :

— **Livraison unique :** une seule livraison est autorisée par commande. L'identifiant `orderId` est unique afin d'éviter les doublons lors de la réception répétée de `OrderConfirmed`.
— **Commande confirmée :** une livraison ne peut être créée que pour une commande confirmée.

— **Affectation du transporteur :** seul un utilisateur existant, actif et possédant le rôle `TRANSPORTEUR` peut être affecté.
— **Modification :** l'adresse et la date prévue peuvent être modifiées uniquement lorsque la livraison est en statut `PENDING` ou `ASSIGNED`.
— **Annulation :** elle est autorisée uniquement lorsque la livraison est en statut `PENDING` ou `ASSIGNED`.
— **Historique :** chaque changement de statut crée automatiquement un nouvel événement `TrackingEvent`.
— **Transition invalide :** toute transition non autorisée est refusée avec le code `409 - Conflict`.
— **Confirmation de réception :** elle est possible uniquement lorsque la livraison est en statut `IN_TRANSIT`. Le statut devient alors `DELIVERED` et la date effective est enregistrée.
— **Échec de livraison :** lorsque la livraison échoue, le statut devient `FAILED` et le motif de l'échec est enregistré.

2.7 MS5– Gestion des cooperatives

2.7.1 Responsabilité

MS5 assure la gestion des besoins des clients et l'organisation des collectes par les responsables de coopératives. Il permet aux clients d'exprimer leurs besoins, aux responsables de publier des annonces de collecte et aux agriculteurs de proposer des quantités issues de leurs récoltes. Il dispose de sa propre base MySQL et d'une API REST, et communique avec MS1 pour les utilisateurs et MS2 pour les produits et récoltes, tout en publiant les événements métier via un broker.

2.7.2 Technologies utilisées

Les technologies utilisées pour le développement de MS1 sont :

— **Java 17** : langage de programmation.
— **Spring Boot** : développement du microservice.
— **Spring Web (REST)** : création des API REST.
— **Spring Data JPA / Hibernate** : gestion de la persistance.
— **MySQL** : système de gestion de base de données.
— **Spring Security** : sécurisation des API.

— **Maven** : gestion des dépendances et compilation.
— **Postman** : test des API REST.
— **Docker** : conteneurisation du microservice.

2.7.3 Choix de la base de données : MySQL

MS5 utilise MySQL, une base de données relationnelle. Ce choix est justifié par les raisons suivantes :

— Permet de gérer les relations entre les entités `BesoinClient`, `ResponsableCooperative`, `AnnonceCollecte` et `PropositionRecolte`.
— Supporte les transactions afin de garantir la cohérence entre les propositions, les quantités acceptées et le statut des besoins.
— Le type `DECIMAL(12,2)` permet de gérer les quantités avec précision, en correspondance avec `BigDecimal`.
— S'intègre facilement avec Spring Boot, Spring Data JPA et Hibernate.
— Chaque microservice possède sa propre base de données, sans clé étrangère SQL vers MS1 ou MS2.

2.7.4 Fonctionnalités

1. **Gestion des besoins clients – Client (Commerçant)**
   — Créer un besoin en précisant le produit, la quantité demandée et la date souhaitée.
   — Consulter ses besoins.
   — Suivre le statut du besoin : en attente, pris en charge, en collecte, satisfait ou annulé.
2. **Gestion des coopératives – Responsable de coopérative**
   — Enregistrer les informations de la coopérative et l'associer à son compte MS1.
   — Activer ou désactiver le responsable.
3. **Gestion des annonces de collecte – Responsable de coopérative**
   — Prendre en charge un besoin et publier une annonce précisant la quantité objectif et la date limite.
   — Consulter les annonces ouvertes et suivre les quantités acceptées.
   — Clôturer ou annuler une annonce.
4. **Gestion des propositions de récolte – Agriculteur / Responsable**
   — Proposer une quantité issue d'une récolte existante, vérifiée auprès de MS2.

— Accepter, accepter partiellement ou refuser une proposition.
— Enregistrer les quantités réellement reçues et suivre les livraisons.

2.7.5 Description des principales entités et énumérations

Le microservice MS5 – Cooperative Management repose sur les entités et les énumérations suivantes :

— **BesoinClient :** Représente le besoin d'un client en un produit agricole, avec la quantité demandée et la date souhaitée.
— **ResponsableCooperative :** Représente le responsable d'une coopérative, lié à un compte MS1 et à son dépôt de réception.
— **AnnonceCollecte :** Représente une annonce publiée pour collecter la quantité demandée auprès des agriculteurs.
— **PropositionRecolte :** Représente une quantité proposée par un agriculteur à partir d'une récolte, avec les quantités acceptée et réellement reçue.
— **StatutBesoin :** Définit les états possibles d'un besoin : en attente, pris en charge, en collecte, satisfait ou annulé.
— **StatutAnnonce :** Définit les états possibles d'une annonce : ouverte, objectif atteint, clôturée ou annulée.
— **StatutProposition :** Définit les états possibles d'une proposition : en attente, acceptée, partiellement acceptée, refusée, reçue ou partiellement reçue.

Règle métier : Un besoin est considéré comme satisfait uniquement lorsque la quantité réellement reçue couvre la quantité demandée, et non sur la base des seules quantités acceptées.

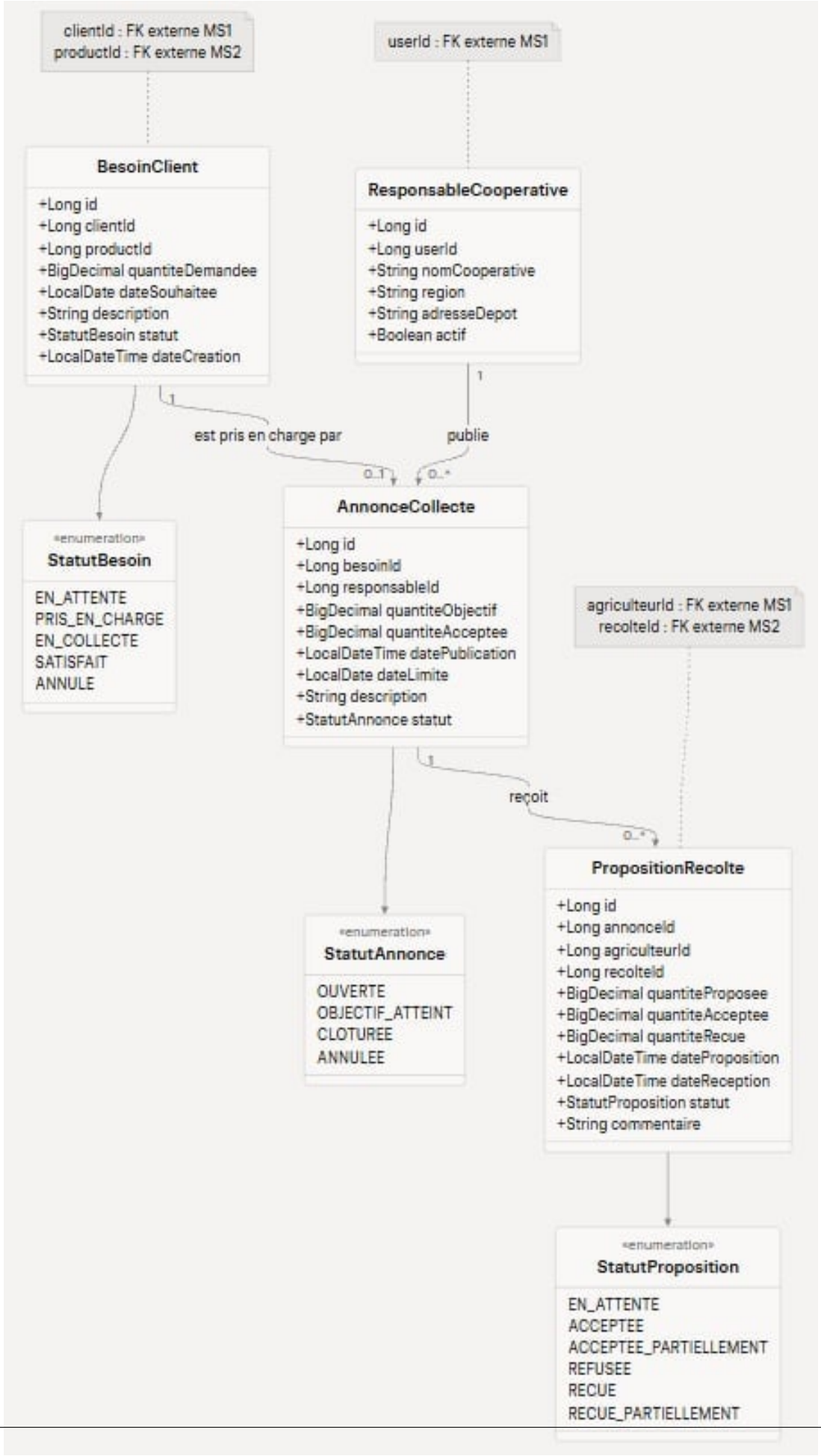


Figure 2.8 – Diagramme de classes du microservice MS5

2.7.6 Exemples d'API REST

Le système expose plusieurs API REST permettant la gestion des besoins clients, l'organisation des collectes et le suivi des propositions de récolte des agriculteurs.

Méthode	**Endpoint**	**Rôle / usage**	**Code HTTP**
POST	`/api/besoins`	Créer un besoin client.	201 Created, 400 Bad Request
POST	`/api/annonces-collecte`	Prendre en charge un besoin et publier une annonce de collecte.	201 Created, 409 Conflict
GET	`/api/annonces-collecte/{id}`	Consulter les détails d'une annonce.	200 OK, 404 Not Found
POST	`/api/annonces-collecte/{id}/propositions`	Soumettre une proposition de récolte par un agriculteur.	201 Created, 503 Service Unavailable
PATCH	`/api/propositions/{id}/decision`	Accepter, accepter partiellement ou refuser une proposition.	200 OK, 409 Conflict
PATCH	`/api/propositions/{id}/reception`	Enregistrer la quantité réellement reçue.	200 OK, 400 Bad Request

2.7.7 Interactions avec les autres microservices

MS5 – Cooperative Management interagit avec MS1 et MS2 pour vérifier les utilisateurs, les produits et les récoltes, ainsi que pour réserver les quantités proposées par les agriculteurs.

2.7.7.1 Services appelants et appelés

Le tableau suivant présente les données échangées entre MS5 et les autres microservices :

Microservice	Sens	Données échangées	Mode
MS1 – User Management	MS5 → MS1	MS5 transmet `clientId`, `userId` ou `agriculteurId`. MS1 retourne l'existence du compte, son statut et son rôle.	REST / OpenFeign
MS2 – Agricultural Management	MS5 → MS2	MS5 transmet `productId`, `recolteId`, `agriculteurId` et la quantité. MS2 retourne l'existence, le propriétaire, la disponibilité et la confirmation de réservation.	REST / OpenFeign
MS2 – Agricultural Management	MS5 → MS2	Événement `QuantiteRecolteRecue` contenant `recolteId` et `quantiteRecue`, publié après la réception effective.	RabbitMQ

Table 2.17 – Données échangées entre MS5 et les autres microservices

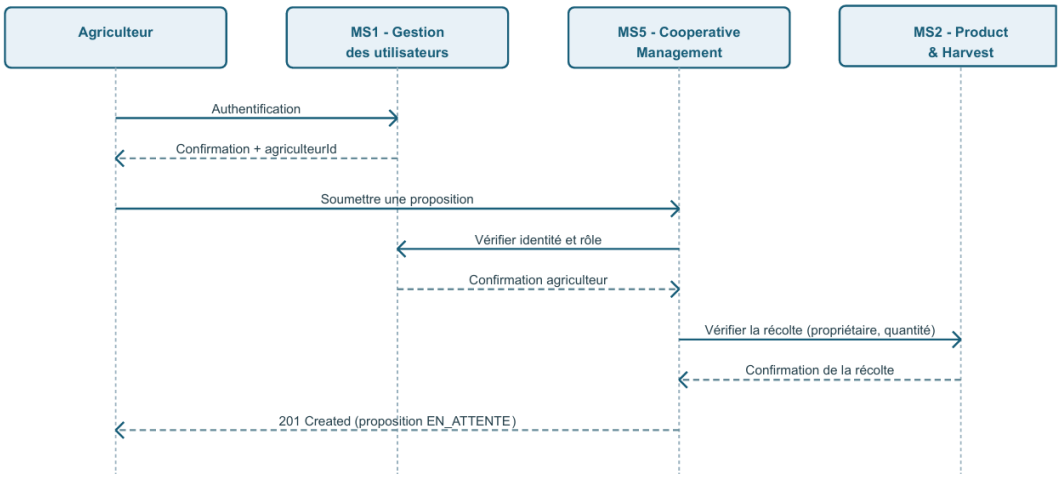


Figure 2.9 – Diagramme d'interaction du microservice MS5 avec MS1 et MS2

2.7.7.2 Mode de communication

Communication synchrone : REST avec OpenFeign

MS5 utilise une communication REST synchrone avec MS1 et MS2 pour effectuer les vérifications nécessaires :

- **Avec MS1 :** vérifier l'existence du compte, son statut et son rôle avant la création d'un besoin, d'une annonce ou d'une proposition.
- **Avec MS2 :** vérifier l'existence du produit et de la récolte, leur propriétaire et leur disponibilité, puis réserver la quantité lors de l'acceptation d'une proposition.

Ces vérifications sont synchrones, car elles conditionnent la validité des opérations métier.

Communication asynchrone : RabbitMQ

Après la réception effective d'une quantité, MS5 publie l'événement `QuantiteRecolteRecue`, consommé par MS2 afin de mettre à jour les informations de la récolte.

Cette communication permet de synchroniser les données sans bloquer le traitement de MS5.

2.7.7.3 Comportement prévu en cas d'indisponibilité

Les comportements prévus en cas de panne sont les suivants :

— **MS1 indisponible :** MS5 retourne une erreur `503 - Service Unavailable` et n'effectue aucune création nécessitant une vérification non réalisée.
— **MS2 indisponible :** MS5 interrompt l'opération nécessitant la vérification ou la réservation de la récolte et retourne une erreur `503 - Service Unavailable`. Aucune proposition ne doit être acceptée sans confirmation de MS2.
— **RabbitMQ indisponible :** l'événement est conservé dans une table Outbox au sein de la même transaction, puis republié après le rétablissement du broker grâce aux mécanismes de Retry.
— **Échec du traitement d'un événement :** des mécanismes d'idempotence et une Dead Letter Queue (DLQ) permettent d'éviter les doublons et de gérer les événements non traités.

Des mécanismes de Timeout, Retry et Circuit Breaker protègent également les appels REST contre les indisponibilités.

Prérequis côté MS2 : La communication nécessite une API REST de réservation des quantités ainsi qu'un mécanisme de consommation de l'événement `QuantiteRecolteRecue`.

2.8 MS6– Contrôle qualité et suivi des certifications agricoles

2.8.1 Responsabilité

MS6 assure le contrôle de la qualité des récoltes agricoles ainsi que la délivrance et le suivi des certifications. Il permet au responsable qualité et certifications de vérifier la conformité des récoltes, de gérer les certifications agricoles et de générer des rapports de contrôle et de certification.

2.8.2 Technologies utilisées

Les technologies utilisées pour le développement de MS6 sont :

— **Java 17** : langage de programmation.
— **Spring Boot** : développement du microservice.
— **Spring Web (REST)** : création des API REST.
— **Spring Data JPA / Hibernate** : gestion de la persistance.
— **H2** : système de gestion de base de données.
— **Spring Security** : sécurisation des API.
— **OpenFeign** : communication synchrone avec les autres microservices.
— **RabbitMQ** : communication asynchrone par événements.
— **Maven** : gestion des dépendances et compilation.
— **Postman** : test des API REST.
— **Docker** : conteneurisation du microservice.

2.8.3 Choix de la base de données : H2 Database

MS6 utilise H2 comme système de gestion de base de données relationnelle. Ce choix est justifié par les raisons suivantes :

— **Simplicité :** H2 est une base de données légère, facile à configurer et à utiliser.
— **Intégration :** elle s'intègre facilement avec Spring Boot et ne nécessite pas de serveur externe.
— **Adaptation :** elle prend en charge SQL et permet de gérer les contrôles qualité, les certifications et les rapports.
— **Transactions :** elle permet de garantir la cohérence des opérations effectuées sur les données.

2.8.4 Fonctionnalités

1. **Gestion des contrôles qualité – Responsable qualité et certifications**
   — Ajouter un contrôle qualité à une récolte.
   — Modifier ou annuler un contrôle.
   — Consulter les contrôles et leur historique.
2. **Génération des rapports de contrôle qualité – Responsable qualité et certifications**

   - Générer automatiquement un rapport à partir des résultats des contrôles.
   - Consulter les rapports générés.
   - Exporter les rapports au format PDF.

3. **Gestion des certifications agricoles – Responsable qualité et certifications**
   - Créer une certification et l'associer à une récolte conforme.
   - Modifier ou archiver une certification.
   - Consulter et rechercher les certifications.
4. **Génération des rapports de certification – Responsable qualité et certifications**
   - Générer des rapports à partir des informations des certifications.
   - Consulter les rapports générés.
   - Exporter les rapports au format PDF.

2.8.5 Description des principales entités et énumérations

Le microservice MS6 – Quality & Certification Management repose sur les entités et les énumérations suivantes :

- **ControleQualite :** représente un contrôle effectué sur une récolte afin de vérifier sa qualité et sa conformité.
- **Certification :** représente une certification délivrée à une récolte ayant satisfait aux contrôles qualité.
- **Rapport :** représente un document généré à partir des données des contrôles qualité ou des certifications.
- **ResponsableQualiteCertification :** représente la référence du responsable qui effectue les opérations dans MS6. Ses informations personnelles et ses rôles sont gérés par MS1.
- **TypeControle :** définit les types de contrôles possibles : réception, stockage, avant commercialisation et expédition.
- **DecisionQualite :** définit le résultat du contrôle : conforme ou non conforme.
- **StatutControle :** définit les états d'un contrôle : en cours, terminé ou annulé.
- **StatutCertification :** définit les états d'une certification : validée, expirée ou archivée.
- **TypeRapport :** distingue les rapports de contrôle qualité des rapports de certification.

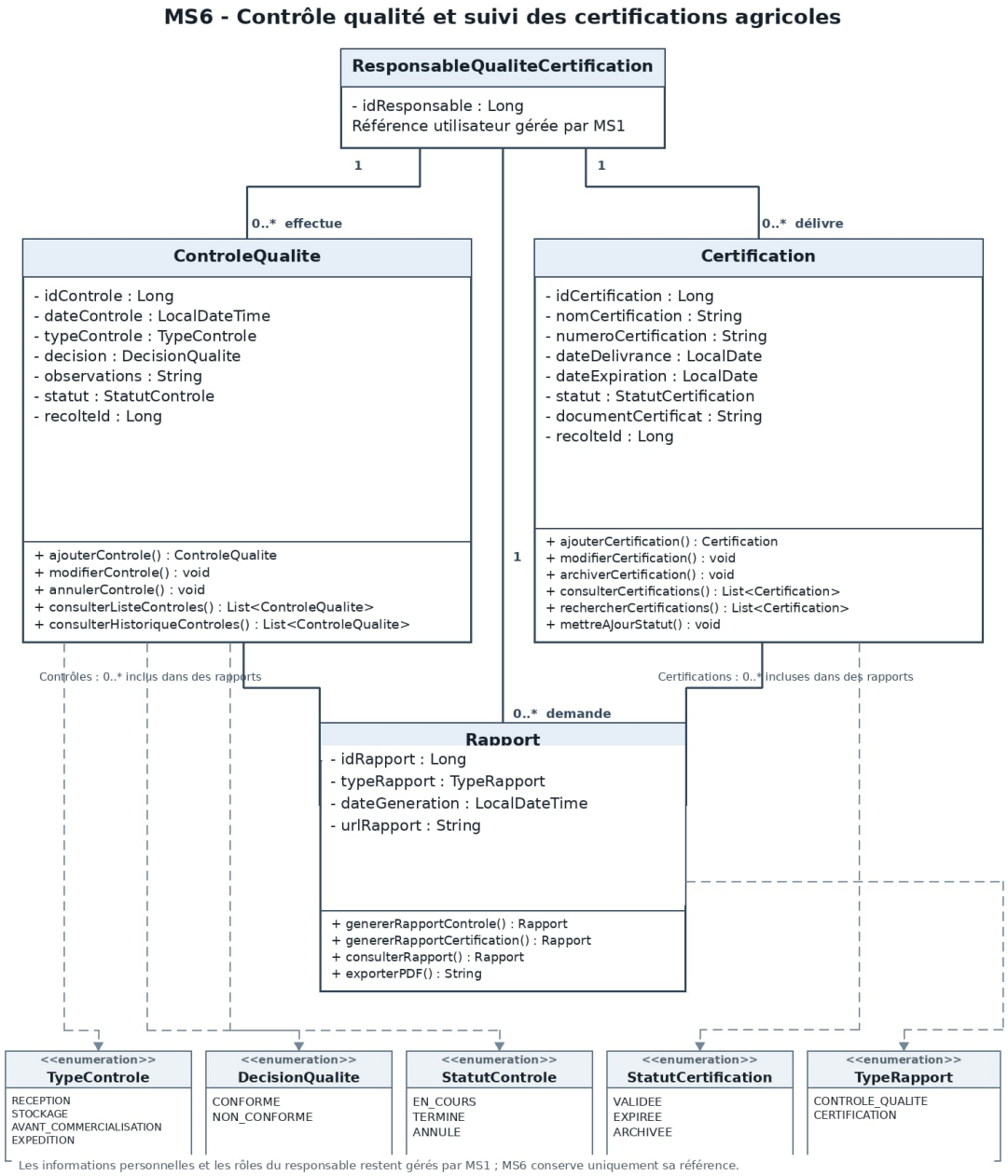


Figure 2.10 – Diagramme de classes du microservice MS6

2.8.6 Exemples d'API REST

MS6 expose plusieurs API REST permettant de gérer les contrôles qualité, les certifications agricoles et les rapports associés.

Méthode	Endpoint	Rôle / usage	Codes HTTP
POST	`/api/qualite-agricole/controles`	Ajouter un contrôle qualité à une récolte.	201, 400, 401, 403, 404

POST	`/api/qualite-agricole/certifications`	Créer une certification pour une récolte conforme.	201, 400, 401, 403, 404
GET	`/api/qualite-agricole/certifications`	Consulter et rechercher les certifications.	200, 401, 403
POST	`/api/qualite-agricole/rapports`	Générer un rapport de contrôle qualité ou de certification.	201, 400, 401, 403
GET	`/api/qualite-agricole/rapports/{id}/telechargement`	download	200, 401, 403, 404

2.8.7 Interactions avec les autres microservices

MS6 – Quality & Certification Management communique principalement avec MS1 et MS2 afin de vérifier les autorisations des utilisateurs et les informations des récoltes avant d'effectuer les opérations de contrôle qualité et de certification.

2.8.7.1 Services appelants et appelés

Le tableau suivant présente les échanges entre MS6 et les autres microservices :

Microservice	**Sens**	**Données échangées**	**Mode**
MS2 – Product & Harvest Management	MS6 → MS2	MS6 envoie `recolteId`. MS2 retourne les informations de la récolte et confirme son existence.	REST / OpenFeign
MS1 – User Management	MS6 → MS1	MS6 transmet `userId` extrait du JWT. MS1 retourne le profil de l'utilisateur et son rôle métier.	REST / OpenFeign
MS2 – Product & Harvest Management	MS6 → MS2	Événement `CertificationIssued` contenant `certificationId`, `recolteId`, `statut`, `dateDelivrance` et `dateExpiration`.	RabbitMQ

Table 2.20 – Données échangées entre MS6 et les autres microservices

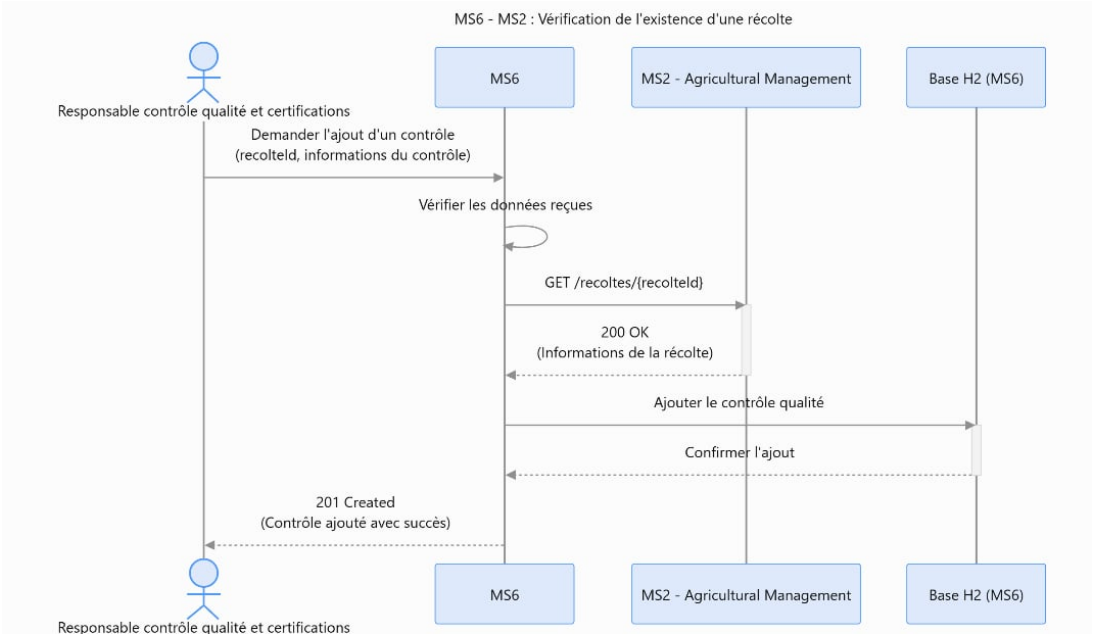


FIGURE 2.11 – Diagramme d'interaction du microservice MS6 avec MS1 et MS2

2.8.7.2 Mode de communication

Communication synchrone : REST avec OpenFeign

MS6 utilise une communication REST synchrone avec MS2 et MS1 pour effectuer les vérifications nécessaires avant certaines opérations métier.

— **MS6 avec MS2 – Vérification d'une récolte :**

Lorsqu'un responsable qualité souhaite ajouter un contrôle à une récolte, MS6 interroge MS2 à travers l'endpoint `GET /api/recoltes/{id}` afin de vérifier l'existence de cette récolte.

MS6 attend la réponse de MS2 avant d'enregistrer le contrôle qualité dans sa base de données H2.

— **MS6 avec MS1 – Vérification des autorisations :**

Avant de créer une certification, MS6 vérifie l'authentification de l'utilisateur et son rôle métier.

Le JWT est d'abord validé localement. MS6 récupère ensuite le `userId` et interroge MS1 à travers l'endpoint `GET /api/users/{id}` pour vérifier le profil et les autorisations du responsable.

Cette communication permet à MS6 d'obtenir une réponse immédiate avant de réaliser les opérations concernées.

Communication asynchrone : RabbitMQ

Après la création d'une certification, MS6 publie un événement métier destiné à MS2 :

— `CertificationIssued` : contient l'identifiant de la certification, celui de la récolte, son statut, sa date de délivrance et sa date d'expiration.
— **Traitement par MS2 :** MS2 reçoit cet événement et met à jour un indicateur associé à la récolte afin de signaler qu'elle possède une certification.

Les informations détaillées des contrôles qualité et des certifications restent gérées exclusivement par MS6.

Cette communication permet à MS6 de notifier MS2 sans attendre la fin de son traitement, tout en préservant l'indépendance des deux microservices.

2.8.7.3 Comportement prévu en cas d'indisponibilité

Les comportements prévus en cas de panne ou d'erreur de communication sont les suivants :

— **MS1 indisponible :** MS6 ne procède pas à la création d'une certification nécessitant la vérification du profil et des autorisations. Il retourne une erreur `503 - Service Unavailable` ou `504 - Gateway Timeout`.
— **MS2 indisponible :** MS6 interrompt l'opération nécessitant la vérification de la récolte. Il retourne une erreur `503 - Service Unavailable` ou `504 - Gateway Timeout`. Aucun contrôle qualité ou aucune certification nécessitant cette vérification ne doit être enregistré sans confirmation de MS2.
— **JWT invalide ou expiré :** MS6 refuse la requête avec le code `401 - Unauthorized`.
— **Rôle insuffisant :** lorsque l'utilisateur ne possède pas le rôle de responsable qualité et certifications, l'opération est refusée avec le code `403 - Forbidden`.
— **RabbitMQ indisponible :** l'événement `CertificationIssued` est conservé temporairement dans une table Outbox de MS6, puis republié après le rétablissement du broker grâce aux mécanismes de Retry.
— **Échec du traitement d'un événement :** MS2 utilise des mécanismes de Retry, d'idempotence et une Dead Letter Queue (DLQ) afin de garantir un traitement fiable et d'éviter les doublons.

2.9 MS7– Gestion de la relation client

2.9.1 Responsabilité

MS7 assure la gestion de la relation client à travers les avis, les notes et les réclamations. Il permet aux clients d'évaluer les agriculteurs après une commande livrée, de déposer des réclamations et de suivre leur traitement jusqu'à leur clôture.

2.9.2 Technologies utilisées

Les technologies utilisées pour le développement de MS7 sont :

— **Node.js** : environnement d'exécution JavaScript côté serveur.
— **NestJS :** framework backend permettant de développer des API REST selon une architecture modulaire.
— **MongoDB** : système de gestion de base de données NoSQL.
— **Mongoose** : gestion des collections et des modèles de données.

2.9.3 Choix de la base de données : MongoDB

MS7 utilise MongoDB, une base de données NoSQL orientée documents. Ce choix est justifié par les raisons suivantes :

— **Flexibilité :** permet de stocker les avis et les réclamations avec des structures de données flexibles.
— **Gestion des messages :** facilite le stockage de l'historique des échanges liés aux réclamations.
— **Évolutivité :** permet de gérer l'augmentation du nombre d'avis et de réclamations.
— **Performance :** permet un accès rapide aux avis, aux réclamations et à leurs messages grâce aux index.

Les principales collections utilisées sont :

— `reviews` : stockage des avis des clients concernant les agriculteurs.
— `complaints` : stockage des réclamations.
— `complaintMessages` : stockage des messages et des réponses associés aux réclamations.

2.9.4 Fonctionnalités

1. **Gestion des avis – Client**
   - Noter un agriculteur après une commande livrée.
   - Ajouter un commentaire sur son expérience.
   - Modifier ou supprimer son propre avis.
   - Consulter les avis et la note moyenne d'un agriculteur.
2. **Gestion des réclamations – Client**
   - Créer une réclamation associée à une commande.
   - Décrire le problème rencontré.
   - Consulter les détails et le statut de sa réclamation.
   - Ajouter des messages et suivre son traitement.
3. **Traitement des réclamations – Administrateur / Service client**
   - Consulter les réclamations reçues.
   - Répondre aux réclamations des clients.
   - Consulter l'historique des échanges.
   - Modifier le statut et clôturer les réclamations.

2.9.5 Description des principales entités

Le microservice MS7 – Customer Relations Management repose sur les entités suivantes :

- **Review :** représente un avis associé à un client et à un agriculteur. Il contient la note, le commentaire, la référence de la commande et la date de création.
- **Complaint :** représente une réclamation liée à un client et à une commande. Elle contient le sujet, la description, le statut et les dates.
- **ComplaintMessage :** représente un message ou une réponse ajouté à l'historique d'une réclamation. Il contient le contenu, l'auteur et la date d'envoi.

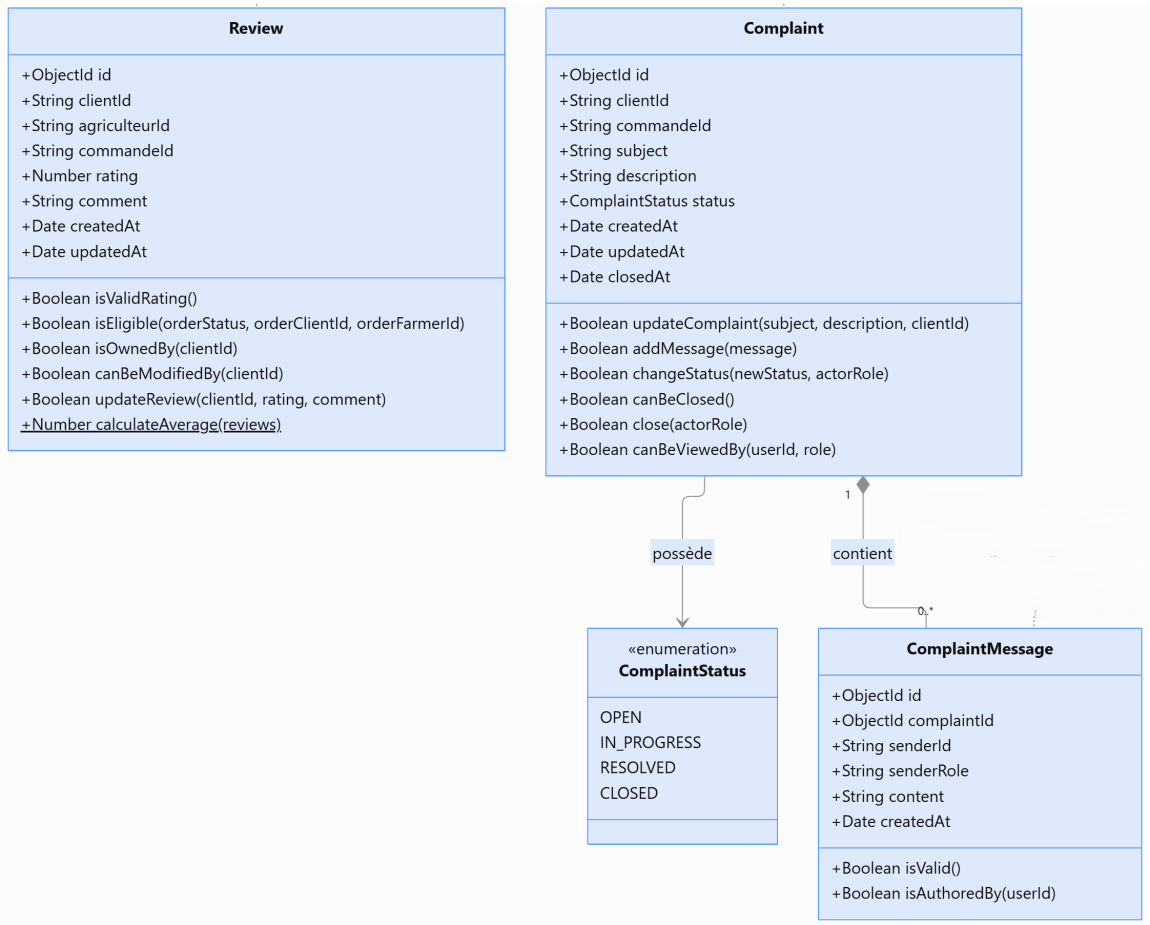


Figure 2.12 – Diagramme de classes du microservice MS7

2.9.6 Exemples d'API REST

MS7 expose plusieurs API REST permettant de gérer les avis des clients et les réclamations, ainsi que leur suivi et leur traitement.

Méthode	Endpoint	Rôle / usage	Code HTTP
POST	`/api/reviews`	Ajouter un avis sur un agriculteur.	201 Created, 400 Bad Request
GET	`/api/reviews/agriculteur/{id}`	Consulter les avis et la note moyenne d'un agriculteur.	200 OK
POST	`/api/complaints`	Créer une réclamation associée à une commande.	201 Created, 400 Bad Request

GET	`/api/complaints/mine`	Consulter les réclamations du client connecté et leur statut.	200 OK, 401 Unauthorized
PATCH	`/api/complaints/{id}/status`	Modifier le statut ou clôturer une réclamation après son traitement.	200 OK, 403 Forbidden, 409 Conflict

2.9.7 Interactions avec les autres microservices

MS7 – Customer Relations Management interagit principalement avec MS1 et MS3 pour identifier les clients, vérifier leurs autorisations et valider les commandes associées aux avis et aux réclamations.

2.9.7.1 Services appelants et appelés

Le tableau suivant présente les données échangées entre MS7 et les autres microservices :

Microservice	**Sens**	**Données échangées**	**Mode**
MS1 – User Management	MS7 → MS1	Identifiant `clientId` extrait du JWT. MS1 retourne les informations nécessaires du profil et confirme l'identité de l'utilisateur.	REST / HTTP
MS3 – Order Management	MS7 → MS3	MS7 transmet `commandeId` et `clientId`. MS3 retourne les informations de la commande, son propriétaire, son statut et l'identifiant `agriculteurId`.	REST / HTTP

Table 2.23 – Données échangées entre MS7 et les autres microservices

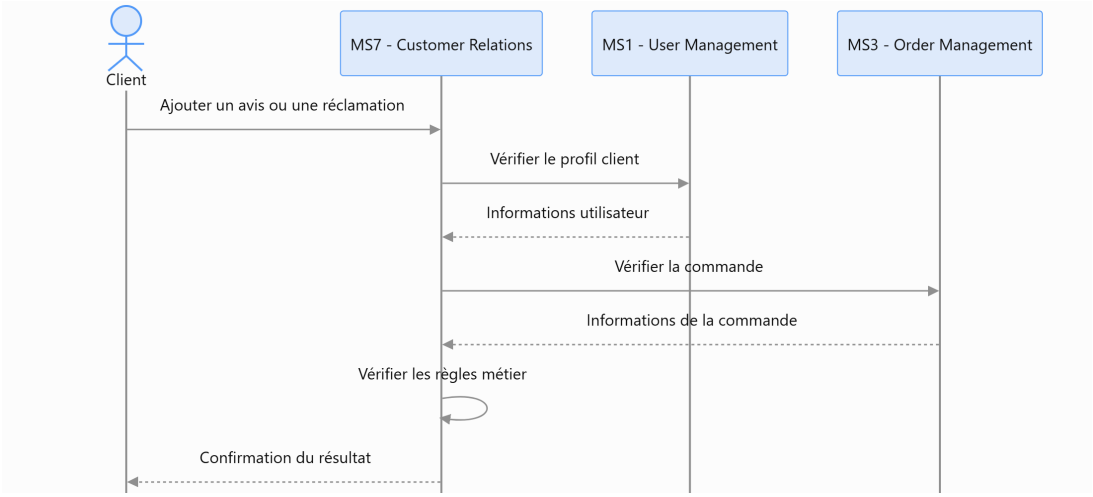


Figure 2.13 – Diagramme d'interaction du microservice MS7 avec MS1 et MS3

2.9.7.2 Mode de communication

Communication synchrone : REST

MS7 utilise une communication REST synchrone avec MS1 et MS3 pour effectuer les vérifications nécessaires avant certaines opérations métier.

— **MS7 avec MS1 – Vérification du profil :**

  MS7 récupère le `clientId` à partir du JWT et vérifie l'identité et les informations du profil auprès de MS1 lorsque cela est nécessaire.

— **MS7 avec MS3 – Vérification des commandes :**

  Avant d'enregistrer un avis, MS7 interroge MS3 pour vérifier l'existence de la commande, son propriétaire, son statut et l'agriculteur concerné.

  Un avis est accepté uniquement si :

  — La commande existe.
  — La commande appartient au client connecté.
  — La commande possède le statut `DELIVERED`.
  — L'agriculteur évalué correspond à celui associé à la commande.

  Cette vérification permet également de contrôler les informations d'une commande associée à une réclamation.

Cette communication permet à MS7 d'obtenir une réponse immédiate avant d'effectuer les opérations concernées.

2.9.7.3 Comportement prévu en cas d'indisponibilité

Les comportements prévus en cas de panne ou d'erreur de communication sont les suivants :

— **MS1 indisponible :** MS7 interrompt l'opération nécessitant la vérification du profil et retourne une erreur `503 - Service Unavailable` ou `504 - Gateway Timeout`.
— **MS3 indisponible :** MS7 effectue un nombre limité de nouvelles tentatives (Retry). Si MS3 reste indisponible, le mécanisme Circuit Breaker peut interrompre temporairement les appels.

  Dans ce cas, MS7 annule temporairement l'opération de création de l'avis et retourne une erreur `503 - Service Unavailable`. Aucun avis n'est enregistré tant que les informations de la commande ne sont pas vérifiées.

— **Utilisateur non autorisé :** si le JWT est invalide, MS7 retourne `401 - Unauthorized`. Si l'utilisateur ne possède pas le rôle requis, l'accès est refusé avec `403 - Forbidden`.
— **Commande non conforme :** si la commande n'appartient pas au client connecté, n'est pas livrée ou ne correspond pas à l'agriculteur évalué, la création de l'avis est refusée.

2.10 Conclusion

Dans ce chapitre, nous avons présenté le découpage d'AgriLink en sept microservices, chacun responsable d'un domaine fonctionnel spécifique. Cette architecture repose sur des services indépendants qui communiquent via REST et RabbitMQ, avec une base de données propre à chaque service.

Ce découpage facilite la maintenance, l'évolution et l'extensibilité de la plateforme AgriLink.

CHAPITRE 3

ARCHITECTURE TECHNIQUE ET GLOBALE

3.1 Introduction

Ce chapitre présente l'architecture technique globale de la plateforme AgriLink ainsi que les principaux composants assurant son fonctionnement.

Il met en évidence l'organisation du frontend, des composants d'infrastructure et des microservices métier, ainsi que les mécanismes permettant leur intégration.

3.2 Vue globale de l'architecture

L'architecture d'AgriLink repose sur plusieurs composants collaborant pour assurer le fonctionnement de la plateforme.

Elle comprend une interface Angular, une API Gateway, un système d'authentification Keycloak, des composants Spring Cloud et sept microservices métier.

Chaque microservice dispose de sa propre base de données. Les échanges entre les services utilisent les mécanismes de communication présentés au chapitre précédent.

3.2.1 Diagramme d'architecture globale

La figure suivante présente l'organisation générale des composants techniques et fonctionnels de la plateforme AgriLink, ainsi que leurs principales relations.

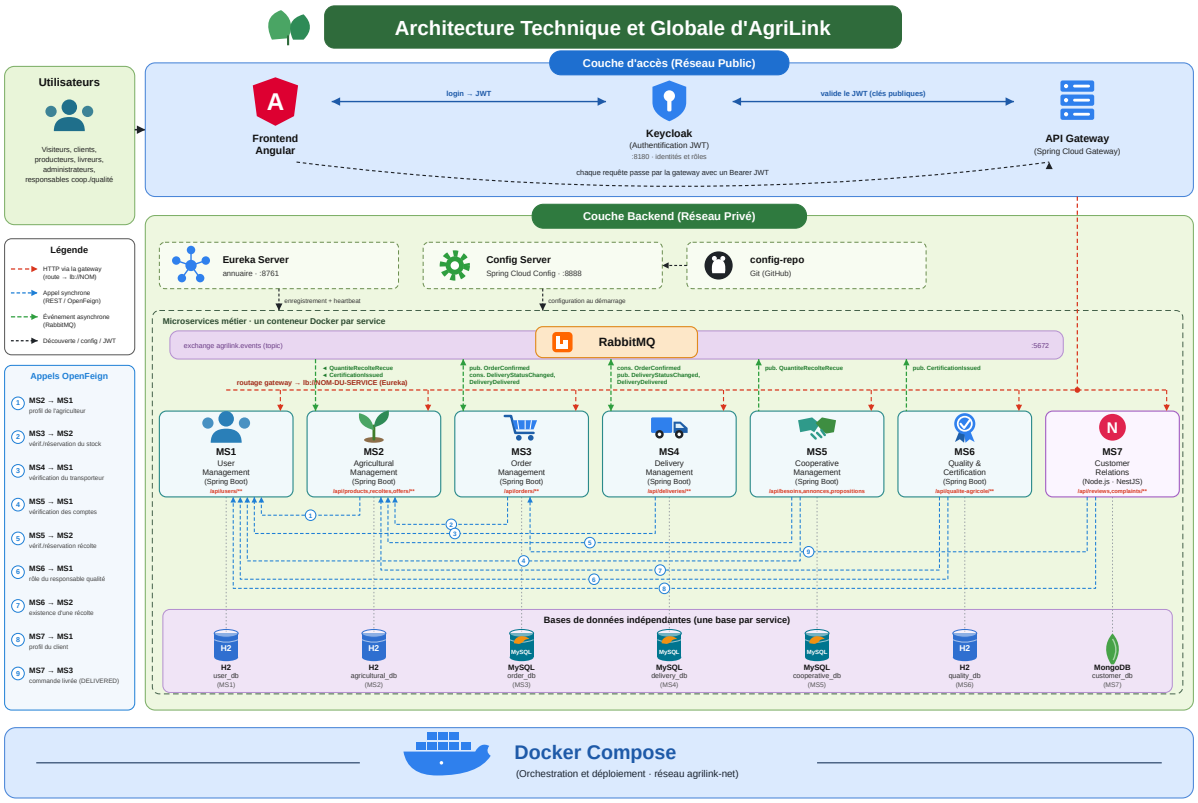


FIGURE 3.1 – Architecture technique et globale de la plateforme AgriLink

3.2.2 Principe général de fonctionnement

Le fonctionnement global de la plateforme suit les étapes suivantes :

1. L'utilisateur accède à la plateforme à travers l'interface Angular.
2. Il s'authentifie auprès de Keycloak, qui délivre un jeton d'accès JWT après vérification de son identité.
3. Le frontend transmet les requêtes à l'API Gateway en incluant le jeton JWT.
4. L'API Gateway identifie le microservice destinataire grâce aux informations fournies par Eureka.
5. Le microservice concerné traite la requête et accède à ses propres données. Si nécessaire, il communique avec les autres services à travers REST/OpenFeign ou RabbitMQ.

3.3 Description des composants techniques

3.3.1 Frontend Angular

Angular constitue la couche de présentation de la plateforme. Il permet aux différents acteurs d'accéder aux fonctionnalités d'AgriLink à travers une interface web.

Il assure la navigation, l'affichage des données et la communication avec le backend à travers l'API Gateway.

3.3.2 API Gateway

L'API Gateway représente le point d'entrée unique du backend. Elle reçoit les requêtes provenant du frontend et les redirige vers les microservices concernés.

Elle permet également de centraliser le routage et de participer à la sécurisation des accès grâce à la vérification des jetons JWT.

3.3.3 Eureka Discovery Server

Eureka est un registre de découverte de services fourni par Spring Cloud Netflix.

Il permet aux microservices de s'enregistrer et de découvrir dynamiquement les autres services, sans dépendre directement de leurs adresses IP ou de leurs ports.

L'API Gateway utilise ce mécanisme pour identifier les services destinataires des requêtes.

3.3.4 Spring Cloud Config Server

Spring Cloud Config Server permet de centraliser les fichiers de configuration des différents composants.

Il facilite la gestion des paramètres de connexion, des ports et des configurations propres aux différents environnements.

Cette centralisation simplifie la maintenance de la plateforme.

3.3.5 Keycloak : gestion de l'identité et de l'authentification

Keycloak est un système de gestion des identités et des accès. Il prend en charge l'authentification des utilisateurs, la gestion des comptes, des mots de passe et des rôles.

Après une authentification réussie, il délivre un jeton JWT utilisé pour accéder aux ressources protégées.

Son intégration avec MS1 repose sur une séparation des responsabilités : Keycloak gère l'identité et la sécurité, tandis que MS1 conserve les informations du profil métier.

Le lien entre les deux composants est assuré par l'attribut `keycloakId`, correspondant à l'identifiant `sub` contenu dans le JWT.

3.3.6 Microservices métier

La couche métier est composée de sept microservices indépendants, chacun responsable d'un domaine fonctionnel de la plateforme.

Leurs responsabilités, leurs entités, leurs technologies et leurs interactions sont détaillées dans le chapitre 2.

Dans l'architecture globale, ils collaborent à travers les mécanismes de communication prévus et conservent l'indépendance de leurs données.

3.4 Conteneurisation et déploiement

Docker est prévu pour faciliter l'exécution et le déploiement des différents composants de la plateforme.

Chaque microservice pourra être exécuté dans un conteneur indépendant. Docker Compose permettra de configurer et de démarrer les différents services dans un environnement commun.

Cette approche facilite :

— L'isolation des environnements d'exécution ;
— Le démarrage et la gestion des composants ;
— La configuration de l'environnement de développement ;
— Les tests et le déploiement de la plateforme.

Les composants concernés comprennent le frontend, l'API Gateway, Keycloak, Eureka, Config Server, RabbitMQ, les sept microservices et leurs bases de données respectives.

3.5 Gestion du code source et organisation du dépôt GitHub

Le projet AgriLink utilise Git et GitHub pour la gestion du code source et la collaboration entre les membres de l'équipe.

Git permet de suivre les modifications apportées au projet, de conserver l'historique des versions et de faciliter le travail collaboratif.

Le dépôt GitHub est organisé en plusieurs répertoires afin de séparer les différentes parties de la plateforme.

3.5.1 Structure du dépôt

Le dépôt principal `AgriLink-4DAS_I3` est organisé de la manière suivante :

```
AgriLink-4DAS_I3/
|
|-- Backend/
|   |-- agricultural-management-service/
|   |-- api-gateway/
|   |-- cooperative-management-service/
|   |-- customer-relations-service/
|   |-- delivery-service/
|   |-- discovery-server/
|   |-- order-service/
|   |-- quality-certification-service/
|   |-- user-management-service/
|   |-- README.md
|
|-- Frontend/
|
|-- Documentation/
|
|-- Infrastructure/
|
|-- README.md
```

Les principaux répertoires sont organisés comme suit :

— **Backend :** regroupe les sept microservices métier, ainsi que les composants techniques API Gateway et Discovery Server.
— **Frontend :** contient la partie interface utilisateur de la plateforme.
— **Documentation :** regroupe les documents et les livrables liés au projet.
— **Infrastructure :** regroupe les éléments liés à l'infrastructure technique du projet.
— **README.md :** présente le projet et fournit les informations générales nécessaires à sa compréhension.

Des fichiers README sont également prévus dans le Backend et les différents services afin de présenter leur organisation et leurs informations spécifiques.

Cette organisation facilite la navigation dans le dépôt, la répartition du travail et la maintenance du projet.

3.6 Conclusion

Dans ce chapitre, nous avons présenté l'architecture technique globale de la plateforme AgriLink ainsi que ses principaux composants.

L'architecture repose sur Angular pour l'interface utilisateur, une API Gateway pour le routage, Keycloak pour l'authentification et les composants Spring Cloud pour la gestion de l'infrastructure.

Les sept microservices constituent la couche métier de la plateforme. Docker et Docker Compose complètent cette organisation en facilitant l'exécution et le déploiement des différents composants.

Cette architecture fournit une base organisée et évolutive pour la réalisation de la plateforme AgriLink.